

题型：都是大题、主要写算法，和作业题类似，一共九道，每一章都涉及到，简单（ppt 例题和作业 90%剩下的题干稍作修改），明确的说作业里有原题（分治那一章设计一个算法 需要稍微说明一下 融合时大概是怎么样的复杂度，只有这里涉及到了）

## Ch01 稳定匹配

### 什么是稳定匹配、优先表、找稳定匹配、GS 算法

问题描述：有 A、B 两方，按己方意愿制定优先表，对两方进行匹配，争取达到稳定匹配。

稳定匹配：不含有不稳定因素的、完美的匹配就是稳定的。



## 1.1 问题

- 稳定匹配
- 不稳定因素
- 给定一个完美匹配 $S$ ，在 $S$ 中存在两个对 $(m, w)$ 和 $(m', w')$ ，如果 $m$ 更偏爱 $w'$ 而不爱 $w$ ，而且 $w'$ 更偏爱 $m$ 而不爱 $m'$ 。  
---称 $(m, w')$ 是一个相对于 $S$ 的**不稳定因素**：  
 $(m, w')$ 不属于 $S$ ，但是 $m$ 和 $w'$ 双方都偏爱另一方而不爱他们在 $S$ 中的伴侣。

完美匹配&优先表：



## 1.1 问题

- 一个**完美匹配** $S'$ 是具有如下性质的匹配： $M$ 的每个成员和 $W$ 的每个成员恰好出现在 $S'$ 的一个队里。
- **优先**的概念：每个男人 $m \in M$ 对所有的女人排名，如果 $m$ 给 $w$ 的排名高于 $w'$ ，称  $m$  偏爱 $w$ 超过 $w'$ 。
- 于是每个男人对女人有一个排名->**优先表**；类似的每个女人也有一个优先表。

邀请拒绝算法：



- 初始，每个人都是自由的。一个**自由**的男人 $m$ 选择他的优先表上排名最高的女人 $w$ ，发起邀请，那么  $(m, w)$  进入**中间状态：约会**。
- 如果又有另一个男人 $m'$ 发起邀请，那么女人 $w$ 决定，选择 $m$ ，还是 $m'$ 。如果 $m$ 优先，那么约会状态不变。否则 $(m', w)$ 变成约会状态， $m$ 变成自由状态。
- 循环往复；最后，当没有人处于自由状态，那么所有的约会将被定为最后的结果，返回最终的匹配。

伪代码：

- 邀请-拒绝算法. [Gale-Shapley 1962]  
找到稳定匹配符合直觉的算法

```
Initialize each person to be free.
while (some man is free and hasn't proposed to every woman) {
    Choose such a man m
    w = 1st woman on m's list to whom m has not yet proposed
    if (w is free)
        assign m and w to be engaged
    else if (w prefers m to m')
        assign m and w to be engaged, and m' to be free
    else
        w rejects m
}
```

性质:

1、自由 def&结果是完美的

- 命题4 如果男人m在算法执行的某点是自由的，那么存在一个他还没有发出过邀请的女人。
- 命题5 终止时，G-S算法返回的集合S是一个完美匹配。

2、稳定匹配证明:

- 命题6 考虑G-S算法的一次执行，它返回一个集合S, 那么S是一个稳定匹配。
- 证明: 假设S中存在一个不稳定因素,  $(m, w), (m', w')$ ; 但是m偏爱 $w'$ ,  $w'$ 偏爱m. 那么m最后一次邀请向w发出; 在此之前, m向 $w'$ 一定发出过邀请, 但是被 $w'$ 拒绝, 那么 $w'$ 一定选择了比m更好的对象, 无论如何 $w'$ 都不可能与 $m'$ 配对。矛盾。

3、特点

- 命题1  $w$ 从接受第一次邀请开始保持约会状态，与她约会的一系列伴侣(依照 $w$ 的优先表)越来越好。
- 命题2  $m$ 提出邀请的一系列女人(按照 $m$ 的优先表)变得越来越差。

- 命题3 G-S算法在至多 $n^2$ 次While循环的迭代后终止。

- 证明：需要定义一个逐步进展的度量。

单个自由人的数目不合适；

参与约会的对数也不合适；

定义 $P(t)$ : 迭代 $t$ 结束时， $m$ 已经向 $w$ 发出过邀请的那些 $(m,w)$ 的集合。

可知 $P(t)$ 大小严格递增。且 $(m,w)$ 只存在 $n^2$ 种可能。

最佳有效伴侣：G-S 每次执行的结果都能得到最佳有效伴侣配对的集合。

- 如果存在一个稳定匹配包含了 $(m,w)$ 对，我们就说女人 $w$ 是男人 $m$ 的有效伴侣。如果 $w$ 是 $m$ 的有效伴侣，且没有别的在 $m$ 的排名中比 $w$ 更高的女人是他的有效伴侣，那么 $w$ 就是 $m$ 的最佳有效伴侣，记为 $best(m)$ 。

- 现在定义 $S^* = \{m, best(m) : m \in M\}$

- 命题7 G-S算法的每次执行都得到集合 $S^*$

最差有效伴侣：

- 类似的，如果存在一个稳定匹配包含了 $(m,w)$ 对，我们就说男人 $m$ 是女人 $w$ 的有效伴侣。如果 $w$ 是 $m$ 的有效伴侣，且没有别的在 $w$ 排名中比 $m$ 更低的男人是她的有效伴侣，那么 $m$ 就是 $w$ 的最差有效伴侣，记为 $worst(w)$ 。



- 命题8 在稳定匹配 $S^*$ 中每个女人与她最差的有效伴侣配对。
- 暗示了一种现象：  
对于任何输入，G-S算法中发出邀请的一方(根据他们的优先表)以**最佳可能**的稳定匹配结束；而另外一方却以**最差可能**的稳定匹配结束。

作业&答案：

## Ch02：算法分析基础

算法复杂度的分析（上界、下界、紧界、对于一些问题怎么找界、给算法知道界是怎样的）

效率 def:

- 寻找效率的定义：与平台无关，实例无关，并且随着输入规模的增长是可以预测的

多项式时间算法：

一个算法被称为是**多项式时间**的如果满足如下的性质：当算法输入的规模增长时，算法的运行时间是多项式有界的。也就是：存在常数 $c > 0$ ,  $d > 0$ , 使得对于每一个问题输入的规模 $N$ , 算法的运行都能够在  $cN^d$  步骤内完成。

- 提出效率定义(3) 如果一个算法有多项式运行时间，称为这个算法是有效的。

- 来自于数学形式和经验证据
- 但并不绝对反映真实的运行时间
- 符合实际情况的合理近似

符号：

- **Upper bounds.**  $T(n)$  is  $O(f(n))$  if there exist constants  $c > 0$  and  $n_0 \geq 0$  such that for all  $n \geq n_0$  we have  $T(n) \leq c \cdot f(n)$ .
- **Lower bounds.**  $T(n)$  is  $\Omega(f(n))$  if there exist constants  $c > 0$  and  $n_0 \geq 0$  such that for all  $n \geq n_0$  we have  $T(n) \geq c \cdot f(n)$ .
- **Tight bounds.**  $T(n)$  is  $\Theta(f(n))$  if  $T(n)$  is both  $O(f(n))$  and  $\Omega(f(n))$ .

其他记号【参考自“算法导论”（第三版）】

- $o$ 记号表示一个非渐进紧确的上界
- 直观上，当 $n$ 趋近于无穷， $f(n)$ 相对于 $g(n)$ 变得微不足道，即  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$ .
- 比如：  $2n = o(n^2)$ , 但是  $2n^2 \neq o(n^2)$
- $\omega$ 记号表示一个非渐进紧确的下界
- 直观上，  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$ .

13

- 设 $f$ 和 $g$ 是两个函数,  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$  存在, 并且等于某个常数 $c > 0$ , 那么 $f(n) = \Theta(g(n))$ .

- 证明: 根据极限的定义, 存在 $n_0$ , 对所有的 $n \geq n_0$ ,  $f(n) \leq 2cg(n)$  从而推出 $f(n) = O(g(n))$ . 类似的 $f(n) = \Omega(g(n))$ . 根据定义可知命题成立。

复杂度性质:

#### ■ 传递性

- If  $f = O(g)$  and  $g = O(h)$  then  $f = O(h)$ .
- If  $f = \Omega(g)$  and  $g = \Omega(h)$  then  $f = \Omega(h)$ .
- If  $f = \Theta(g)$  and  $g = \Theta(h)$  then  $f = \Theta(h)$ .

#### ■ 可加性

- If  $f = O(h)$  and  $g = O(h)$  then  $f + g = O(h)$ .
- If  $f = \Omega(h)$  and  $g = \Omega(h)$  then  $f + g = \Omega(h)$ .
- If  $f = \Theta(h)$  and  $g = O(h)$  then  $f + g = \Theta(h)$ .

- 命题 令 $f$ 是一个 $d$ 阶多项式,  $a_0 + a_1n + \dots + a_d n^d$ ,  $a_d > 0$ , 那么 $f = \Theta(n^d)$ .

- 命题 对于任何正整数,  $O(\log_a n) = O(\log_b n)$ .

- 命题 对于  $x > 0$ ,  $\log n = O(n^x)$ .

对数函数增长比多项式增长慢!

- 命题 对每个 $r > 1$ 和每个 $d > 0$ ,  $n^d = O(r^n)$ .

多项式增长比指数函数慢!

算法分类：

- 对日常中我们经常碰到的算法问题，按照阶数的大小进行分类
- $O(\log n)$
- $O(n), O(n \log n)$
- $O(n^2), O(n^3), O(n^k)$
- $O(a^n), O(n!)$

(1) 亚线性时间：

## 亚线性时间( $O(\log n)$ )

- 例子：给定一个排好序的含有 $n$ 个数的数组 $A$ ，我们想确定一个给定的数 $p$ 是否属于这个数组。
- 算法要点：每次比较 $p$ 与剩余集合中间元素的值
- $k$ 次试探以后，剩余的区间大小为： $\left(\frac{1}{2}\right)^k n$   
所以选择 $k = \log_2(n)$ ，剩余集合的大小减少到一个常数，最后可在常数时间内直接搜索。

(2) 线性时间：

## 线性时间

- 线性时间  $O(n)$

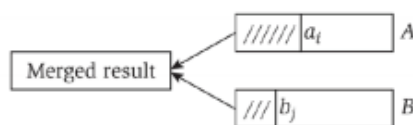
问题：计算 $n$ 个数 $a_1, \dots, a_n$ 中的最大数。

```
max ← a1
for i = 2 to n {
    if (ai > max)
        max ← ai
}
```

## 线性时间

归并两个排好序的数表

- 问题：给定两个数组，每个包含有 $n$ 个数，并且每个按照上升的顺序排列， $\mathbf{A} = a_1, a_2, \dots, a_n$ ； $\mathbf{B} = b_1, b_2, \dots, b_n$ ，把它们归并成按照上升顺序排列的数组。



```
i = 1, j = 1
while (both lists are nonempty) {
    if (a_i ≤ b_j) append a_i to output list and increment i
    else (a_i ≤ b_j) append b_j to output list and increment j
}
append remainder of nonempty list to output list
```

## $O(n \log n)$ 阶时间

- 快速排序(Quicksort)
- FFT(快速傅立叶变换)
- 归并排序，堆排序

## $O(n \log n)$ 阶时间

- 实际中碰到的问题：
- **最大时间间隔问题**：给定一组 $n$ 个时间标签 $x_1, \dots, x_n$ ，一个文件的副本在这些时间到达一个服务器，我们想找出在第一个和最后一个时间标签之间的**最大的**没有文件副本到达的**时间区间**。
- 本质上是一个排序问题：寻找最大的时间间隔

(3)  $N$  次方时间：

## 平方时间

- 问题：假设平面上给定 $n$ 个点，每个点由 $(x, y)$ 坐标指定。需要找最邻近(距离最小)的点对。
- $O(n^2)$ 解决方法：一个个点对尝试

## 立方时间

- 问题：给定集合 $S_1, \dots, S_n$ ，它们都是 $\{1, 2, \dots, n\}$ 的子集，我们想知道这些子集中是否有某对子集是**不相交的**，也就是没有共同的元素。
- 解决思路：对每对集合 $S_i, S_j$ ，确定是否 $S_i, S_j$ 有一个共同的元素。



## 立方时间

```
foreach set  $S_i$  {  
  foreach other set  $S_j$  {  
    foreach element  $p$  of  $S_i$  {  
      determine whether  $p$  also belongs to  $S_j$   
    }  
    if (no element of  $S_i$  belongs to  $S_j$ )  
      report that  $S_i$  and  $S_j$  are disjoint  
  }  
}
```

$O(n^3)$  solution

## $O(n^k)$ 时间

- 问题： 对某个固定常数 $k$ , 我们想知道给定的 $n$ 个结点的输入图是否有一个大小为 $k$ 的独立集。
- 如果用蛮力搜索的方法： 枚举所有 $k$ 个结点的子集，并且对每个子集 $S$ 检查是否存在一条边与 $S$ 中的任意两个元素相交



## $O(n^k)$ 时间

```
foreach subset S of k nodes {  
  check whether S is an independent set  
  if (S is an independent set)  
    report S is an independent set  
}
```

枚举所有 $k$ 个结点的子集:  $\binom{n}{k} = \frac{n(n-1)(n-2)\cdots(n-k+1)}{k(k-1)(k-2)\cdots(2)(1)} \leq \frac{n^k}{k!}$

判断 $k$ 个结点的子集是否独立:  $O(k^2)$

所以蛮力搜索的代价为:  $O(k^2 n^k / k!) = O(n^k)$

(4) 指数时间

## 指数时间(Exponential time)

- 问题: 假设给定一个图, 需要找一个**最大规模的独立集**。
- 解决思路: 蛮力搜索的方法, 检查每一个子集是否是独立集, 是否最大

## 指数时间(Exponential time)

```
S* ←  $\phi$   
foreach subset S of nodes {  
  check whether S is an independent set  
  if (S is largest independent set seen so far)  
    update S* ← S  
}
```

估算计算时间的代价

$$O(n^2 2^n)$$

## n!时间

- 对稳定匹配问题的穷举搜索:  $n!$
- 二分匹配(二部图每边存在 $n$ 个结点)问题  
如果穷举搜索, 代价是 $n!$
- 巡回售货员(TSP: Travelling Salesman Problem)问题: 给定 $n$ 个城市的集合, 每对城市之间都有距离, 什么是访问所有城市的最短旅行? 固定第一个(结束)城市  
穷举代价是 $(n-1)!$

## n!时间

- Stirling公式 (1730)

$$\lim_{n \rightarrow \infty} \frac{n!}{\sqrt{2\pi n} n^n e^{-n}} = 1.$$

$$\text{记成当 } n \rightarrow \infty, n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n.$$

作业&答案:

### CH03 图:

概念: 有向图、无向图、树, 算法: 广度优先、深度优先, 相关应用 (二分性测试、怎么判别是不是二部图, 拓扑排序 (拓扑排序的含义))

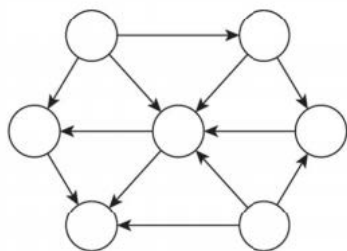
1、概念:

- 无向图  $G = (V, E)$ 
  - $V$  = 顶点
  - $E$  = 边，反映顶点之间的关系
  - 图参数:  $n = |V|, m = |E|$ .

- Def. 无向图  $G = (V, E)$  的一条路径  $P$  定义为如下的结点序列  $v_1, v_2, \dots, v_{k-1}, v_k$ , 其中  $v_i, v_{i+1}$  是  $E$  中的一条边.  $P$  被称为从  $v_1$  到  $v_k$  的路径。
- Def. 如果一条路径所有的结点都是相互不同的, 就称为简单的。
- Def. 如果对于无向图中任意两个顶点  $u, v$ , 都存在一条路径, 那么无向图被称为是连通的。

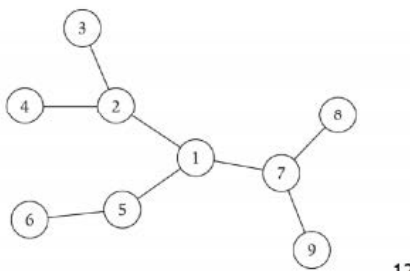
## 有向图中的连通性

- 有向图(Directed graph)  $G = (V, E)$ 
  - Edge  $(u, v)$  : 从  $u$  到  $v$



## 树的定义

- **Def.** 一个无向图被称为是**树**，如果它是连通的而且没有圈。
- **定理.** 设  $G$  是一个具有  $n$  个结点的无向图。下面任意两条都可以推出第三条。
  - $G$  是连通的.
  - $G$  不包含一个圈.
  - $G$  有  $n-1$  条边.

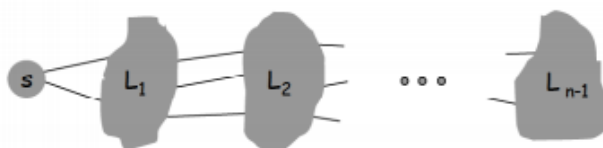


### 2、算法

- a. 广度优先算法 (BFS):

## 广度优先(Breadth-First Search)

- **BFS算法简介.** 从顶点  $s$  开始向各个方向“探索”，把探索到的顶点依次加在各层上。



- **BFS algorithm.**
  - $L_0 = \{s\}$ .
  - $L_1 =$  all neighbors of  $L_0$ .
  - $L_2 =$  all nodes that do not belong to  $L_0$  or  $L_1$ , and that have an edge to a node in  $L_1$ .
  - $L_{i+1} =$  all nodes that do not belong to an earlier layer, and that have an edge to a node in  $L_i$ .

```

· BFS(s):
  · 置Discovered[s]=true, 其他v, 置Discovered[v]=false
  · 初始化L[0], 单个元素s构成
  · 置层计数器i=0
  · 置BFS树T=空集
  · While L[i]非空
    · 初始化一个空表L[i+1]
    · For 每个结点L[i]中结点u
      · 考虑每条关联到u的边(u, v)
      · If Discovered[v]=false then
        · 置 Discovered[v]=true
        · 把边(u, v) 加到树T上
        · 把v加到表L[i+1]
      · Endif
    · Endfor
    · 层计数器加1
  · Endwhile

```

性质:

## BFS性质

- 定理3. 11 如果图是邻接表给出, BFS算法的上述实现将以 $O(m+n)$ 时间运行。
- Pf. 考虑结点  $u$ , 存在  $\deg(u)$  条与 $u$ 相连的边  $(u, v)$ , 所以处理所有边的时间是  $\sum_{u \in V} \deg(u) = 2m$ ; 此外对于顶点, 还要  $O(n)$ 的额外时间来管理数组Discovered.

- 定理 **BFS**算法产生的层  $L_i$  就是到源点 $s$ 距离为 $i$ 的顶点集合. 存在一条 $s$ 到 $t$ 的路径当且仅当  $t$  出现在某一层中.
- 定理 设 $T$ 是图 $G = (V, E)$ 的一棵宽度优先搜索树,  $(x, y)$ 是 $G$ 中的一条边.那么  $x$  ,  $y$  所属的层数至多相差 1.

## BFS & DFS

- **BFS**: 从  $s$ 开始出发探查, 按照距离分层;
- **DFS**: 从 $s$ 开始出发探查, 方式更 “冲动”;
- **BFS**: 一般用**队列**来维护即将被处理的结点;
- **DFS**: 用**栈**来维护即将被处理的结点;

b. 深度优先算法:

## DFS

- DFS( $s$ )
- 初始化 $S$ 为具有一个元素 $s$ 的栈
- While  $S$  非空
- 从 $S$ 中取出一个节点 $u$
- If Explored[ $u$ ]=false then
- 置Explored[ $u$ ]=true
- For每条与 $u$ 关联的边( $u, v$ )
- 把 $v$ 加到栈 $S$
- Endfor
- EndIf
- EndWhile



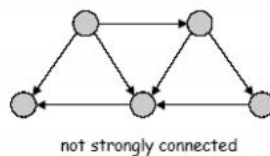
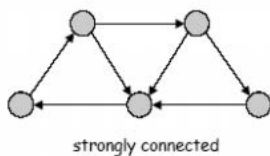
### 3、连通性质：

## 连通性质

- 定理3.8： 对无向图中任两个结点 $s$ 与 $t$ ,它们的连通分支或者相等，或者不相交。
- 定理3.17 对有向图中的任何两个结点 $s$ 与 $t$ ,它们的**强连通**分支或者相等，或者不相交。

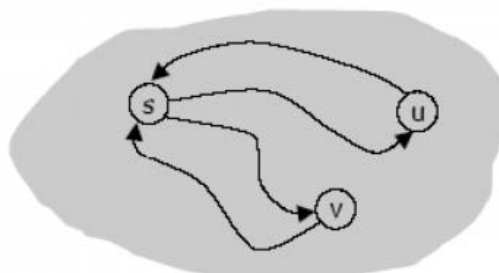
## 强连通性

- Def.  $u$ 和 $v$ 是**相互可达**的，如果彼此之间存在到达对方的路径。
- Def. 如果图中每对结点是相互可达的，那么此图是**强连通**的。



- 引理. 设 $s$ 是图 $G$ 中的任意一个结点。 $G$ 是**强连通**的，当且仅当图中的每个结点能够与 $s$ 相互可达。

Pf.



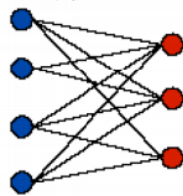
- Pick any node  $s$ .
- Run BFS from  $s$  in  $G$ .
- Run BFS from  $s$  in  $G^{\text{rev}}$ .
- Return true iff all nodes reached in both BFS executions.

存在  $O(m + n)$  的有效算法判别图  $G$  是否强连通。

#### 4、应用

##### a) 二分性测试：

- 二部图是一个图，其中结点集可以划分为  $X$  与  $Y$ ，每条边一端在  $X$  中，另一端在  $Y$  中。
- 直观的，一个图是二部图，如果能对结点着红色和蓝色，使得每条边有一个红端点和一个蓝端点。

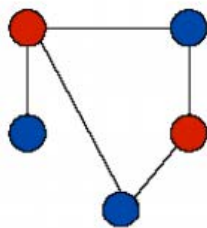


a bipartite graph

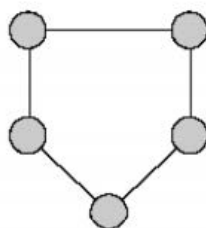
31

## 二部图性质

- 定理 如果一个图是二部图，那么它不可能包含一个奇圈。

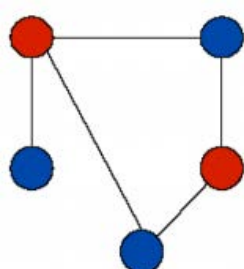


bipartite  
(2-colorable)

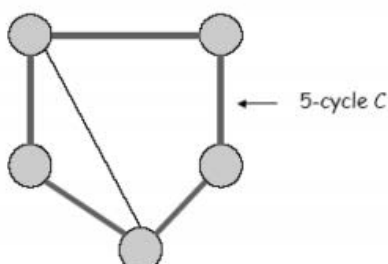


not bipartite  
(not 2-colorable)

- 引理 图 $G$ 是二部图当且仅当图中没有奇圈。



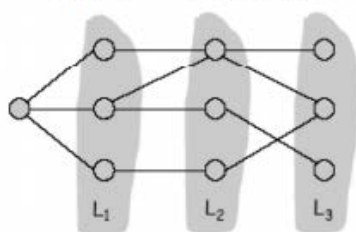
bipartite  
(2-colorable)



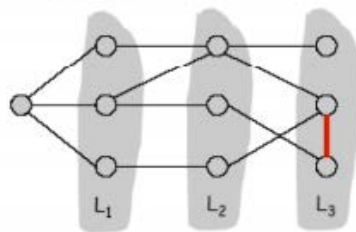
not bipartite  
(not 2-colorable)

## 二部图性质

- 定理 设 $G$ 是一个连通图， $L_0, \dots, L_k$  是从顶点 $s$ 由BFS算法生成的层。那么下面两件事一定恰好成立其一：
  - $G$ 中没有边与同一层的两个结点相交。**这种情况下 $G$ 是二部图，其中偶数层的结点可以着红色，奇数层结点可以着蓝色。
  - $G$ 中有一条边与同一层的两个结点相交。**此种情形下，存在一个奇圈，不可能是二部图。



Case (i)

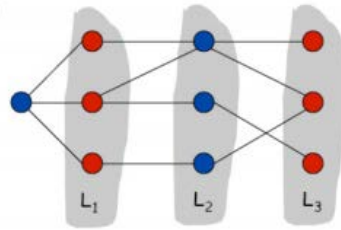


Case (ii)

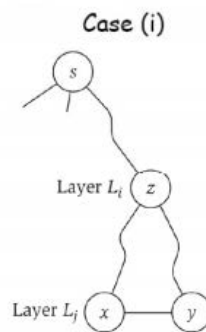
34

## 二部图性质

- Pf. (i) 红蓝交替着色



- (ii) 证明产生一个奇圈



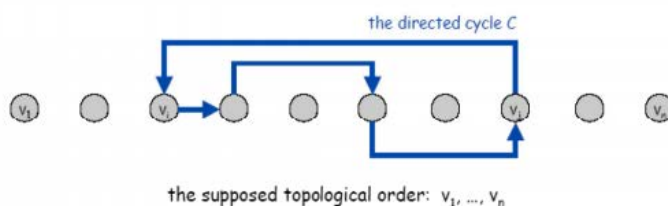
- b) 拓扑排序:

## 3.6 有向无圈图与拓扑排序

- 问题的背景: 存在标记为 $\{1, 2, \dots, n\}$ 任务集, 之间存在先后的依赖性, 是否存在一个有效的执行任务的次序?
- 有向图 $G = (V, E)$ 的一个**拓扑排序**是它的结点作为 $v_1, v_2, \dots, v_n$ 的一个排序, 对每条边 $(v_i, v_j)$ , 都有 $i < j$ .
- Def. **DAG**(Directed Acyclic Graphs): 有向图没有圈

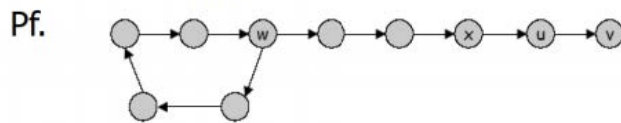
定理: 如果 $G$ 有一个拓扑排序, 那么 $G$ 是一个DAG.

Pf.



## 万事开头难：第一个结点

- 命题：在每一个DAG中，存在一个没有输入边的结点。



定理 如果G是一个DAG, 那么G有一个拓扑排序。

Pf. 归纳证明

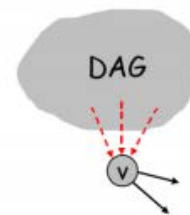
## DAG

### ■ 算法

---

To compute a topological ordering of  $G$ :  
Find a node  $v$  with no incoming edges and order it first  
Delete  $v$  from  $G$   
Recursively compute a topological ordering of  $G - \{v\}$   
and append this order after  $v$

---



定理 上面算法在  $O(m + n)$  时间内找到一个拓扑排序.

Pf. 考虑边逐次递减的代价 $O(m)$ ; 追踪被删除的结点代价 $O(n)$ .

作业&答案:

## CH04: 贪心算法

怎么证明贪心算法最优 (贪心算法领先【每一步都领先】、交换认证), 实例 (区间调度、最小延迟调度、



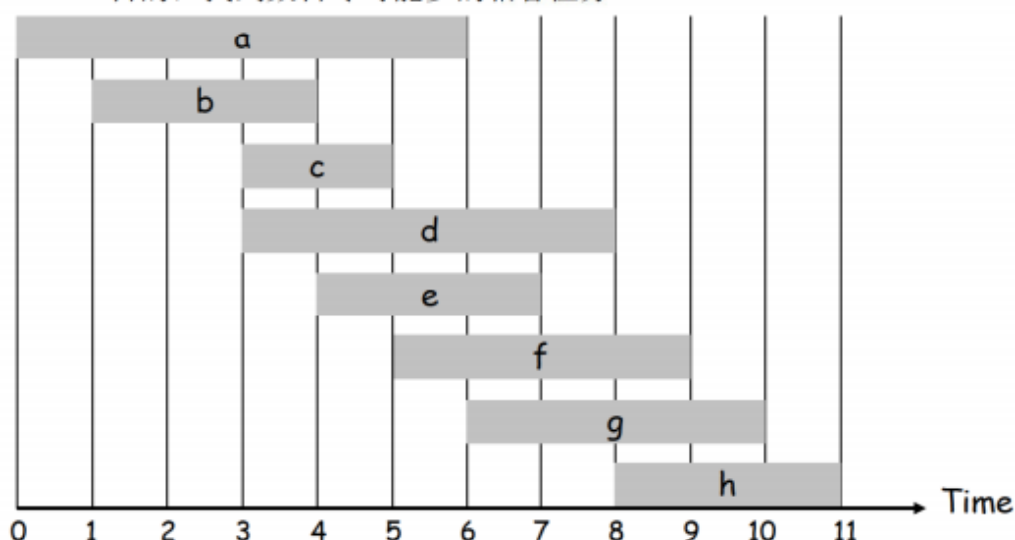
缓存), 最短路径 (迪杰斯特拉) 最小生成树 (Prim、Kruskal) 聚类、哈夫曼编码 (计算频率、做叶子节点, 生成上层节点)

## 1、证明

- 本章逐渐介绍两个基本的方法来证明一个贪心算法对一个问题能够提供一个最优解。
  - ✓ 1. **贪心算法领先**的概念: 每一步都比其他的算法好, 从而证明产生了一个最优解。
  - ✓ 2. **交换论证**: 考虑对这个问题的任何可能解, 逐渐把它转换成由贪心算法找到的解, 而且不影响解的质量。从而证明了贪心法找到了一个至少与其它解一样好的解。

a) 贪心算法领先: (区间调度实例中实现)

- **区间调度问题:**
- 任务 (需求)  $j$  开始时间  $s_j$ , 结束时间  $f_j$ .
- 两个任务是相容的如果任务的时间区间不相交
- 目的: 找到数目尽可能多的相容任务





- 那么最好的方案应该是什么？
- 吸取前面的经验---与开始时间、区间的宽度、最少“冲突”都有一点联系
- 应该选择什么样的规则？
- 我们应该接受最早结束的需求，即 $f(i)$ 尽可能小的需求 $i$ 为第一个需求。这样的好处是资源尽可能早被释放，以便于安排下面的需求。

## 贪心算法

- 初始令 $R$ 是所有需求的集合，设 $A$ 为空
  - While  $R$  非空
    - 选择一个最小结束时间的需求 $i \in R$
    - 把 $i$ 加到 $A$ 中
    - 从 $R$ 中删除与需求 $i$ 不相容的所有需求
  - Endwhile
  - 返回集合 $A$ 作为被接受的需求集合
- 
- 命题4.1 由上面贪心规则返回的集合 $A$ 中的区间都是相容的。
- 
- 需要证明这个解是最优的，为了便于比较，令 $O$ 是一个最优的区间集合。证明的主要思想是贪心算法生成的集合 $A$ “领先”于集合 $O$ 。
  - 目标是为了证明 $|A|=|O|$ 。
  - 引入如下记号：设 $i_1, i_2, \dots, i_k$ 是 $A$ 中的需求，并按照加入到 $A$ 的次序排列， $|A|=k$ 。类似的， $j_1, j_2, \dots, j_m$ 是 $O$ 中的需求。目标是为了证明 $k=m$ 。

- 我们的贪心规则保证：**A**中的每个区间至少与集合**O**中对应的区间结束得一样早。

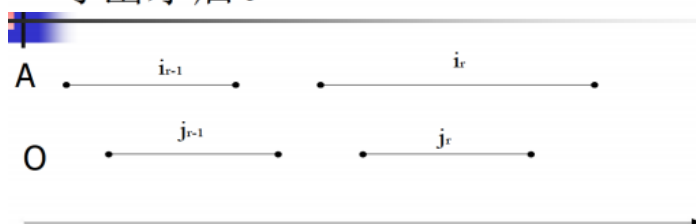
- 命题4.2 对所有的指标 $r \leq k$ , 我们有  $f(i_r) \leq f(j_r)$ .

- 证明：用归纳法

- 对于 $r=1$ , 论断为真。

- 如果 $r>1$ , 根据归纳假设, 假定这个论断对 $r-1$ 为真, 我们将试图证明对 $r$ 也为真。

如下图, 由归纳假设我们知道 $f(i_{r-1}) \leq f(j_{r-1})$ , 为了使算法的第 $r$ 个区间不更早结束, 假设它处于“落后状态”, 即 $f(i_r) > f(j_r)$ , 下面将导出矛盾。



- 从上面可以看出:  $f(j_{r-1}) \leq s(j_r)$ ; 又有  $f(i_{r-1}) \leq f(j_{r-1})$ ; 所以  $f(i_{r-1}) \leq s(j_r)$ . 若  $f(i_r) > f(j_r)$  意味着A集合中应该选择 $j_r$ 而不是 $i_r$ , 矛盾。所以  $f(i_r) \leq f(j_r)$ , 归纳成立。

**定理4.3** 上面的贪心算法返回一个最优的集合**A**.

证明：如果存在一个更好的最优集合**O**满足更多的任务需求。即我们有 $m > k$ . 根据命题4.2, 得到 $f(i_r) \leq f(j_r)$ 。在**O**中存在一个需求 $j_{r+1}$ , 在 $j_r$ 后开始, 从而也在 $i_r$ 后开始. 从贪心算法的步骤中可以看出**A**应该不在 $i_r$ 处停止, 矛盾。

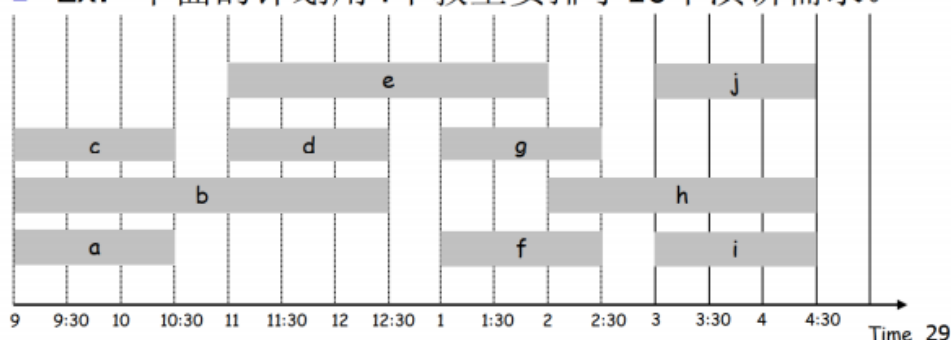
b) 交换论证

i) 最小延迟调度

- 区间划分问题描述.

- 演讲  $j$  从  $s_j$  开始, 在  $f_j$  结束。
- 目标: 寻找尽可能少的教室满足所有的这些演讲需求, 其中同一个教室中没有演讲冲突。

■ Ex: 下面的计划用4个教室安排了10个演讲需求。



- 贪心算法要点: 需求(演讲)按照开始时间排序, 把需求安排到不冲突的教室中。

```
Sort intervals by starting time so that  $s_1 \leq s_2 \leq \dots \leq s_n$ .  
d  $\leftarrow$  0  
  
for j = 1 to n {  
  if (lecture j is compatible with some classroom k)  
    schedule lecture j in classroom k  
  else  
    allocate a new classroom d + 1  
    schedule lecture j in classroom d + 1  
    d  $\leftarrow$  d + 1  
}
```

## 4.2 最小延迟调度

- 问题：我们有单一资源和一组使用资源的 $n$ 个需求，每个需求需要一个时间区间。假定需求 $i$ 更加灵活，不是开始时间与结束时间，需求 $i$ 有一个截至时间 $d_i$ ，要求一个长度为 $t$ 的连续的时间区间。目标是需要尽可能多的满足需求。

### 描述

- 需求  $j$  需要长度为  $t_j$  时间段，截至时间为  $d_j$ .
- 需求  $j$  如果在  $s_j$  开始，那么结束时间是  $f_j = s_j + t_j$ .
- 延迟的定义：  $l_j = \max \{ 0, f_j - d_j \}$ .
- 目标：安排所有的需求，使得计划具有最小的延迟调度：让  $L = \max l_j$  最小.

### ■ 贪心算法(最早截至时间优先)

```
Sort n jobs by deadline so that  $d_1 \leq d_2 \leq \dots \leq d_n$ 

 $t \leftarrow 0$ 
for  $j = 1$  to  $n$ 
    Assign job  $j$  to interval  $[t, t + t_j]$ 
     $s_j \leftarrow t, f_j \leftarrow t + t_j$ 
     $t \leftarrow t + t_j$ 
output intervals  $[s_j, f_j]$ 
```

■ 命题4.7 存在一个没有空闲时间的最优调度

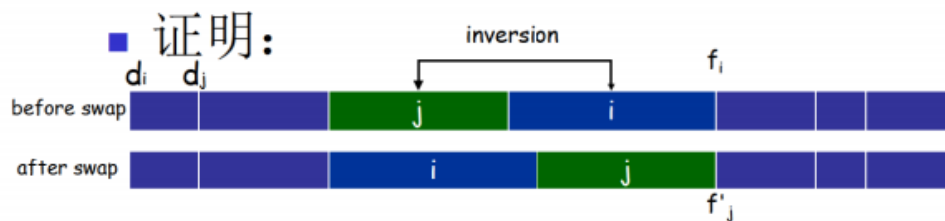
- 思路：开始将考虑一个最优调度 $O$ ,我们的计划是逐步修改 $O$ ,每步保持它的最优性，最终把它转换成一个与贪心算法得到的调度 $A$ 相等的调度。这种类型的分析看作**交换论证**。
  - 尝试如下方式刻画调度：一个调度 $A'$ 有一个**逆序**，如果具有截止时间 $d_i$ 的任务 $i$ 被安排在具有更早截止时间 $d_j < d_i$ 的任务 $j$ 的前边。



命题4.8 所有没有逆序也没有空闲时间的调度有相同的最大延迟。

- 证明：两个不同的调度仅可能在具有相同截止时间的任务安排次序上不一样。这两个不同调度中，具有截止时间 $d$ 的任务全部被连续安排。在这些任务中，**最后一个有着最大的延迟**，而且这个延迟不依赖于这些任务的次序。
- 命题：如果调度中存在一个**相邻**的逆序任务 $i, j$ ，那么交换以后，所得到的新调度方案不会增加最大延迟。





设 $\ell$ 是交换之前的延迟,  $\ell'$ 是交换后的延迟。

■  $\ell'_k = \ell_k$  for all  $k \neq i, j$

■  $\ell'_i \leq \ell_i$

■ 那么

$$\begin{aligned}
 \ell'_j &= f'_j - d_j && \text{(definition)} \\
 &= f_i - d_j && (j \text{ finishes at time } f_i) \\
 &\leq f_i - d_i && (i < j) \\
 &= \ell_i && \text{(definition)}
 \end{aligned}$$

53

■ 定理4.9 存在一个即没有逆序, 也没有空闲时间的最优调度。

■ 证明

(1) 如果调度O有一个逆序, 那么存在一对相邻的逆序 $i, j$ 。

(2) 交换 $i$ 和 $j$ 之后, 我们得到具有减少一个逆序的新调度。

(3) 新的被交换的调度的最大延迟不大于O的最大延迟;

(4) 消除所有逆序。

■ 定理4.10 前面贪心算法产生的调度有最优的最大延迟。

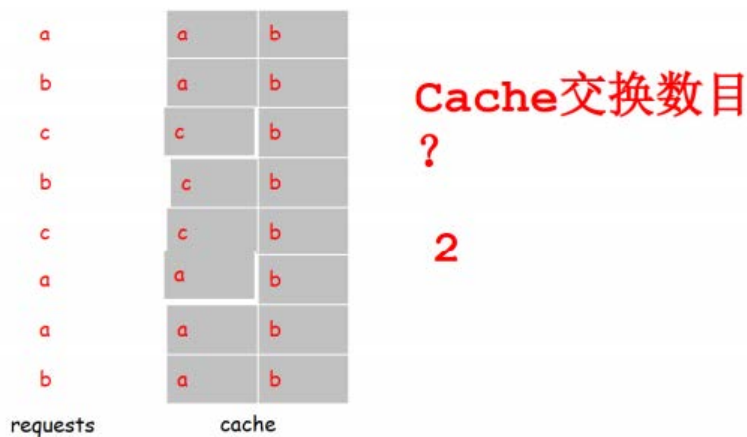
■ 证明: 定理4.9证明存在一个没有逆序的最优调度; 又根据命题4.8, 所有没有逆序的最优调度有着相同的最大延迟。所以贪心算法产生最优解。

ii) 最优高速缓存



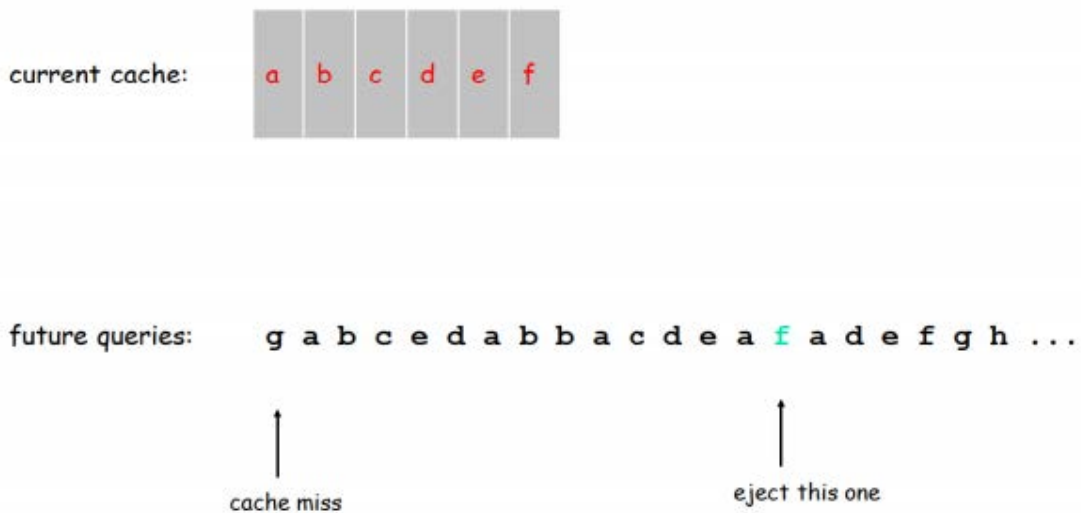
- 问题抽象:
- 主存中有 $n$ 块数据的集合 $U$
- 更快的超高速缓存，一次能够保存 $k < n$ 个数据。从 $U$ 中取出一系列数据项：  
 $D = d_1, d_2, \dots, d_n$ ; 需要决定在每个时刻哪 $k$ 个项保存在超高速缓存中。

- 对某项 $d_i$ ,如果已经在超高速缓存中(访问命中, **cache hit**), 那么访问速度很快; 否则, 需要从主存中调配过来。这时, 需要收回在超高速缓存中别的数据, 为 $d_i$ 空出位置。这种情形叫做**超高速缓存缺失(cache miss)**。我们希望尽可能少出现这些缺失, 少做交换。
- Ex:  $k = 2$ , 初始 cache = ab,  
访问序列: a, b, c, b, c, a, a, b.



- “**最远将来规则**” ( Farthest-in-future )  
 When  $d_i$ 需要被放入超高速缓存  
 收回在**最远的将来**被需要的那个项

## ■ 例子



- Theorem. [Bellady, 1960s] FF (Farthest-in-future) 调度方案是最优调度方案，得到最小的交换次数。

## FF调度分析

- 定义：如果在第*i*步有对*d*的需求，而且*d*不在超高速缓存中，那么考虑在第*i*步只放入项*d*的调度，这种调度被称为简化调度。

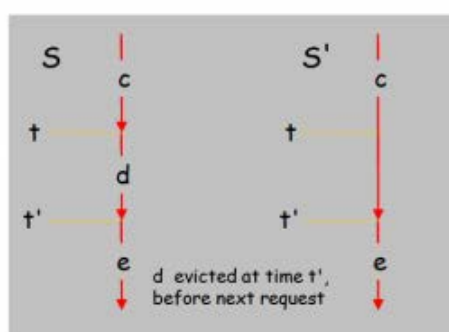
|   |   |   |   |
|---|---|---|---|
| a | a | b | c |
| a | a | x | c |
| c | a | d | c |
| d | a | d | b |
| a | a | c | b |
| b | a | x | b |
| c | a | c | b |
| a | a | b | c |
| a | a | b | c |

an unreduced schedule

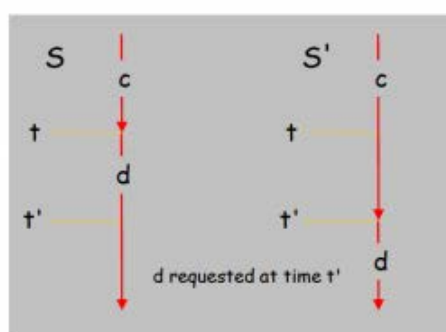
|   |   |   |   |
|---|---|---|---|
| a | a | b | c |
| a | a | b | c |
| c | a | b | c |
| d | a | d | c |
| a | a | d | c |
| b | a | d | b |
| c | a | c | b |
| a | a | c | b |
| a | a | c | b |

a reduced schedule

- **Claim.** 给定任意一个非简化调度 $S$ , 可以将其转换成一个简化调度 $S'$ , 并不增加缓存交换次数.
- **Pf.** 假设 $S$ 在时刻 $t$ , 把 $d$ 带入缓存中交换 $c$ , 尽管这时没有需求。
  - 情形1: 假设在下次需求 $d$ 之前,  $d$ 在 $t'$ 发生交换。
  - 情形2: 假设在时刻 $t'$ 需求 $d$ .



Case 1



Case 2

73

- **定理4.12** 对于某个数 $j$ , 令 $S$ 是在序列中的前 $j$ 项与 $S_{FF}$ 做同样收回决定的简化调度, 那么存在一个在序列中的前 $j+1$ 项与 $S_{FF}$ 做同样收回决定的简化调度 $S'$ , 并且 $S'$ 不比 $S$ 产生更多的交换。

证明: 通过对需求 $j$ 进行归纳 (存在一个最优的简化调度 $s$ , 在前面 $j+1$ 个需求中与 $S_{FF}$ 的缓存调度的策略完全一样)

设 $s$ 是一个简化调度, 与 $S_{FF}$ 在前 $j$ 个调度完全一样。下面我们将构造一个 $s'$ 使得在前 $j+1$ 个需求中与 $S_{FF}$ 完全一样。

- 考虑第 $(j+1)$ 个需求 $d=d_{j+1}$ .
- 既然 $s$ 和与 $S_{FF}$ 在前 $j$ 个调度完全一样, 那么它们在第 $j+1$ 个需求之前缓存中是完全一样的。
- 情形1: ( $d$ 已经在缓存中)  $s' = s$ . 满足所需性质.
- 情形2: ( $d$ 不在缓存中但是 $s$ 和 $S_{FF}$ 交换同样的元素)  $s' = s$ , 满足所需性质.

证明 (续)

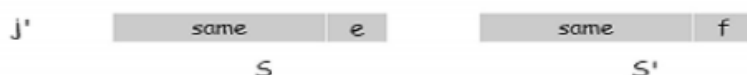
情形3: (d不在缓存中, SFF交换e; S交换f ≠ e)

-于是我们构造s', 与s不同的是, 交换的是e而不是f



现在s' 与SFF在前面j+1个需求都是一样的: 下面我们说明在缓存中保留f不比保留e差。

设j' 是第一次在j+1个需求后s和s' 采用不同的动作 (意味着必须包括e, f的操作), 设g是在时刻j' 需要的元素。



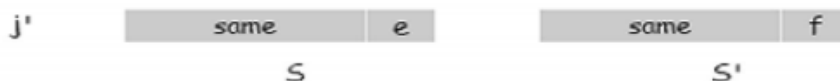
Case 3a: g=e 这是不可能的, 违反了Farthest-In-Future的性质。

Case 3b: g=f. 因为f不可能在s调度的缓存中, 设交换的元素是e'.

-如果e'=e, 注意到s' 中已经有f; 于是此时开始s与s' 有相同的缓存。

- 如果e' ≠ e, s' 用e' 交换e, 此时开始s与s' 有相同的缓存。(尽管s' 此时不是简化调度, 但是可以转换成前j+1步与SFF相同的简化调度)

设j' 是第一次在j+1个需求后s和s' 采用不同的动作 (意味着必须包括e, f的操作), 设g是在时刻j' 需要的元素。



Case 3c: g ≠ e, f. S 必须用 e 交换.

我们让S'用f交换; 此时S和S'具有完全相同的缓存。



- 体现了交换论证的思想，如果 $S'$ 含有“贪心”的性质，那么不比 $S$ 产生更多的交换。
- 这个定理说明可以存在一个**连续的**简化调度优化，变成最后的 $S_{FF}$ .
- 定理4.13  $S_{FF}$ 比其他任何的调度 $S'$ 不产生更多的交换，因此是最优的。

## 2、最短路径问题（dijkstra 算法）

### Dijkstra算法

- 保持一个结点集 $S$ ，如果源点 $s$ 到 $u$ 的最短路径一旦确定，那么 $u$ 就加到 $S$ 中。
- 最开始  $S = \{s\}$ ,  $d(s) = 0$ .
- 循环寻找结点 $v$ ，使得  $\pi(v) = \min_{e=(u,v): u \in S} d(u) + \ell_e$ ,  


  
 从已经探索过的某个结点 $u$ 到 $v$ 的边  $(u, v)$

**最小**，然后把 $v$ 加入 $S$ ，并设  $d(v) = \pi(v)$ .

伪代码：



Dijkstra算法( $G, l$ )

设 $S$ 是被探查的结点集合

对每个 $S$ 中的 $u$ , 存储一个 $d(u)$ ;

初始 $S = \{s\}$  且  $d(s) = 0$

While  $S \neq V$

选择一个结点不在 $S$ 中的结点 $v$ , 使得从 $S$ 到 $v$ 至少  
有一条边连接并且

$d'(v) = \min_{e=(u,v), u \in S} d(u) + l_e$  最小  
将 $v$ 加入 $S$ 并且定义 $d(v) = d'(v)$

Endwhile

- **定理4.14** 算法执行中任意一点的集合 $S$ ,  
对每个 $S$ 中的 $u$ , 路径 $P_u$ 是最短的 $s$ - $u$ 路径。

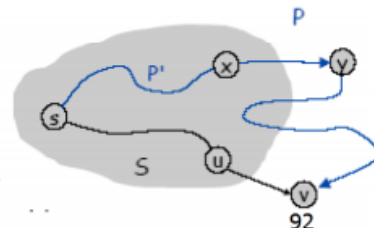
- Pf. 采用归纳法

- $|S| = 1$  易证.

- 假设  $|S| = k \geq 1$  命题成立。

- Let  $v$  be next node added to  $S$ , and let  $u-v$  be the chosen edge.
- The **shortest**  $s$ - $u$  path plus  $(u, v)$  is an  $s$ - $v$  path of length  $\pi(v)$ .
- **Consider any  $s$ - $v$  path  $P$ . We'll see that it's no shorter than  $\pi(v)$ .**
- Let  $x-y$  be the first edge in  $P$  that leaves  $S$ , and let  $P'$  be the subpath to  $x$ .
- $P$  is already too long as soon as it leaves  $S$ .

$$\begin{array}{ccccccc} \ell(P) & \geq & \ell(P') + \ell(x, y) & \geq & d(x) + \ell(x, y) & \geq & \pi(y) \geq \pi(v) \\ \uparrow & & \uparrow & & \uparrow & & \uparrow \\ \text{nonnegative} & & \text{inductive} & & \text{defn of } \pi(y) & & \text{Dijkstra chose } v \\ \text{weights} & & \text{hypothesis} & & & & \text{instead of } y \end{array}$$





- 对于每个“无关顶点”，不做改变
- 对于“相邻顶点”，当 $v$ 被加入“核心”时，对于边 $e = (v, w)$ ，更新临时最短距离  

$$\pi(w) = \min \{ \pi(w), \pi(v) + \ell_e \}.$$
- 提高效率的方法：采用优先队列来存放没有探索的结点，根据 $\pi(v)$ 来排序。

### 3、最小生成树 (MST)

#### ■ 问题抽象：

对于确定的边 $e=(v_i, v_j)$ ，存在一个正的费用 $C_e$ ，问题是对于图 $G=(V, E)$ ，寻找  $T \subseteq E$  使得图 $(V, T)$ 是连通的，而且  $\sum_{e \in T} C_e$  最小。

- 命题4.16 设 $T$ 是满足上面网络设计问题的最小费用解，那么 $(V, T)$ 是一棵树。

算法：

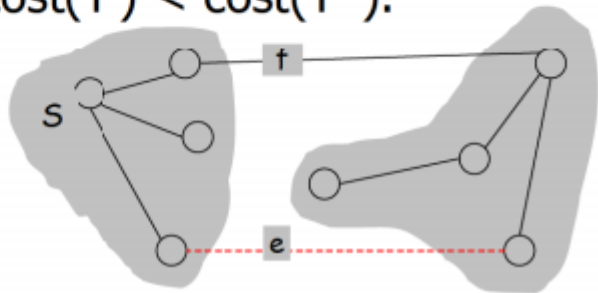
- **Kruskal's algorithm.** 初始  $T = \phi$ . 边按照费用递增次序排列。通过不断插入边来建立一棵生成树：把边  $e$  插入  $T$  只要不构成圈。
- **Reverse-Delete algorithm(反向删除算法).** 初始  $T = E$ . 依照费用递减的次序开始删除边。当达到每条边  $e$  时(从最贵的边开始)，只要这样做不破坏当前图的连通性。
- **Prim's algorithm.** 初始  $S = \{s\}$ , 然后贪心选择的生长树  $T$ . 每步选择一端在  $T$  中, 费用最小的边与  $T$  连接。

(1) 证明:

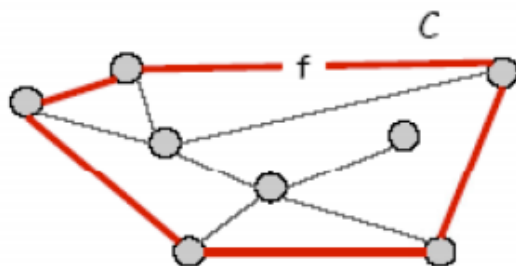
- **定理4.17** 假设所有边的费用都是不等的。令  $S$  是任意节点子集,  $S \neq \phi, V$ , 令边  $e = (u, v)$  是一端在  $S$  中, 另一端在  $V - S$  中的最小费用边。那么每棵最小生成树都包含边  $e$ .

■ Pf. (采用交换论证)

- 假设  $e$  不属于  $T^*$ (最小生成树).
- 增加  $e$  到  $T^*$  形成了一个圈  $C$ .
- 边  $e$  即属于圈  $C$ , 也属于与  $S$  对应的割  $D$ ;  $\Rightarrow$  一定存在另一条边  $f$ , 即属于  $C$  也属于  $D$ .
- $T' = T^* \cup \{e\} - \{f\}$  也是一个生成树.
- 因为  $c_e < c_f$ ,  $\text{cost}(T') < \text{cost}(T^*)$ .
- 导出矛盾. ■



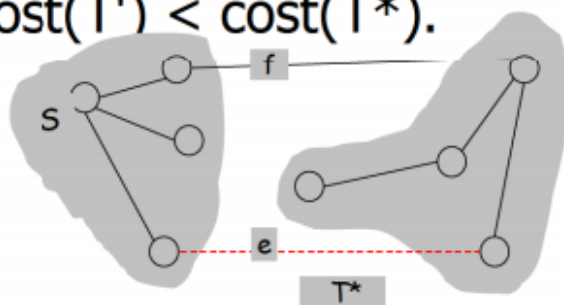
- 定理4.20 假设所有边的费用都是不等的。令  $C$  是  $G$  的一个圈，令边  $f=(v,w)$  是属于  $C$  的最贵的边。那么  $f$  不属于  $G$  的任何最小生成树。



$f$  is not in the MST

110

- 证明：（采用交换论证）假设  $f$  属于  $T^*$ .
  - 把  $f$  从  $T^*$  中删除得到一个集合  $S$ .
  - $f$  即在圈  $C$  中，也在与  $S$  对应的割  $D$  中  $\Rightarrow$
  - 一定存在另一条边  $e$ , 即属于  $C$  也属于  $D$ .
  - $T' = T^* \cup \{e\} - \{f\}$  也是一个生成树.
  - 因为  $c_e < c_f$ ,  $\text{cost}(T') < \text{cost}(T^*)$ .
  - 导出矛盾. ■



1

- 定理4.21 逆删除算法产生  $G$  的一棵最小生成树。
- 证明：最后算法输出  $(V, T)$  是连通的。  
如果  $(V, T)$  包含一个圈  $C$ . 考虑  $C$  上最贵的边  $e$ , 它将是算法遇到的第一条边，移走应该不会破坏图的连通性，与逆删除算法的行为矛盾。

(2) Prim 算法：

- 初始  $S =$  任意结点.
- 每次把与  $S$  对应的割中最小费用边加入  $T$ , 同时把新的探索结点  $u$  加入  $S$ .

伪代码：

```

Prim(G, c) {
  foreach (v ∈ V) a[v] ← ∞
  Initialize an empty priority queue Q
  foreach (v ∈ V) insert v onto Q
  Initialize set of explored nodes S ← ∅

  while (Q is not empty) {
    u ← delete min element from Q
    S ← S ∪ { u }
    foreach (edge e = (u, v) incident to u)
      if ((v ∉ S) and (ce < a[v]))
        decrease priority a[v] to ce
  }
}

```

- 复杂度估计： 采用优先队列（堆）， $O(m \log n)$ .
- 定理4.19 Prim算法产生一棵最小生成树。
- 证明： 对于Prim算法，容易知道它只加了那些属于每棵最小生成树的边。根据定义，**e**是使得一个结点在**S**中而另一个结点在**V-S**中费用最低的边( $\min_{e=(u,v), u \in S} c_e$ )，所以根据割性质4.17，**e**在每一棵最小生成树中。

(3) Kruskal 算法：

- 边按照费用递增的顺序排序.
- **Case 1:** 如果把边 $e$ 加入 $T$ 时构成了一个圈, 那么跳过边 $e$ .
- **Case 2:** 否则, 把边  $e = (u, v)$  加入  $T$  中 (根据割性质), 其中  $S$  是与 $u$ 连通的结点集合。
- 算法代价:  $O(m \log n)$ .

```

Kruskal( $G, c$ ) {
  Sort edges weights so that  $c_1 \leq c_2 \leq \dots \leq c_m$ .
   $T \leftarrow \phi$ 

  foreach ( $u \in V$ ) make a set containing singleton  $u$ 

  for  $i = 1$  to  $m$ 
    ( $u, v$ ) =  $e_i$ 
    if ( $u$  and  $v$  are in different sets) {
       $T \leftarrow T \cup \{e_i\}$ 
      merge the sets containing  $u$  and  $v$ 
    }
  return  $T$ 
}

```

(4) 逆删除算法:

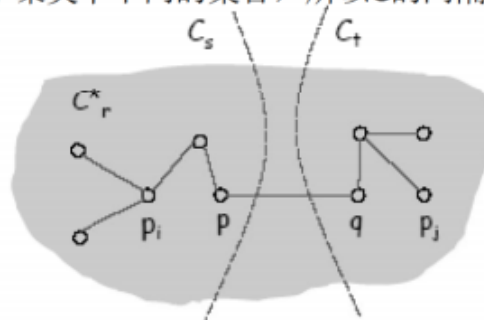


- 定理4.21 逆删除算法产生 $G$ 的一棵最小生成树。
- 证明：最后算法输出 $(V, T)$ 是连通的。  
如果 $(V, T)$ 包含一个圈 $C$ 。考虑 $C$ 上最贵的边 $e$ ，它将是算法遇到的第一条边，移走应该不会破坏图的连通性，与逆删除算法的行为矛盾。

#### 4、聚类

- 也就是，运行Kruskal算法，但是就在它加最后的 $k-1$ 条边之前停止。
- 等价于取整棵最小生成树 $T$ ，然后删除 $k-1$ 条最贵的边，并且定义 $k$ 聚类是所得到的连通分支 $C_1, C_2, \dots, C_k$ 。
- 定理4.26 由删除最小生成树 $T$ 的 $k-1$ 条最贵的边所构成的连通分支 $C^*: C_1^*, C_2^*, \dots, C_k^*$ 组成一个最大间隔的 $k$ 聚类。

- 证明： 设 $C: C_1, \dots, C_k$ 是另外一个聚类
  - $C^*$  的间隔就是第 $k-1$ 条最贵边的边，长度为 $d^*$ .
  - 设  $p_i, p_j$  在  $C^*$ 的同一个聚类 $C_r^*$ 中，但是却在 $C$ 不同的聚类中，比如  $C_s$  与  $C_t$ .
  - 由于 $p_i, p_j$ 属于同一个连通分支 $C_r^*$ ，所以一定在我们停止之前Kruskal算法添加了 $p_i-p_j$ 路径上所有的边。特别的，这里 $p_i-p_j$ 路径上每条边的长度至多是 $d^*$ 。
  - 因为相邻的 $p, q$ 属于聚类中不同的集合，所以 $C$ 的间隔  $\leq d^*$  ■



132

## 5、 Huffman 码

### 前缀码

- 对于一组字母 $S$ 的前缀码是把每个字母  $x \in S$  以下述方式映射到一个0，1序列的函数  $\gamma$ ，对于不同的  $x, y \in S$ ，序列  $\gamma(x)$  不是序列 $\gamma(y)$ 的前缀。

比如  $S = \{a, b, c, d, e\}$ ，定义如下的编码方式

$$\gamma_1(a) = 11, \gamma_1(b) = 01, \gamma_1(c) = 001, \gamma_1(d) = 10, \gamma_1(e) = 000$$

- 最优前缀码

- 假设对每一个字母  $x \in S$ ，文本中等于字母  $x$  的字母频率为  $f_x$ ，字母总量为  $n$ 。

那么编码总的长度为：
$$\sum_{x \in S} n f_x |\gamma(x)| = n \sum_{x \in S} f_x |\gamma(x)|$$

我们的目标使得每个字母的平均位数

$$ABL(\gamma) = \sum_{x \in S} f_x |\gamma(x)| \text{ 最小。}$$

- 1948 Claude Shannon 奠定了编码的信息论基础

- 宏观的结论：

- 如果  $\gamma$  是一种可译编码方案 (Instantaneous code), 那么

$$ABL(\gamma) \geq H_r(f_1, f_2, \dots)$$

$$\text{其中 } H_r(f_1, f_2, \dots) = \sum_{x \in S} f_x \log_r \frac{1}{f_x}$$

- 我们的直观，频率大的字母，对应的编码(树高)短。
- 假设  $u$  与  $v$  是  $T^*$  的树叶，使得  $\text{depth}(u) < \text{depth}(v)$ . 树叶  $u$  对应于  $y \in S$ ，树叶  $v$  对应于  $z \in S$ ，那么  $f_y \geq f_z$ 。

- 证明:  $ABL(T^*) = \sum_{x \in S} f_x \cdot \text{depth}(x)$ , 交换结点  $u$  与  $v$ , 那么总和的改变是  $(\text{depth}(v) - \text{depth}(u))(f_y - f_z)$ , 因为  $\text{depth}(u) < \text{depth}(v)$ , 而这个改变需要是非负数, 所以  $f_y \geq f_z$ .

- 考虑  $T^*$  中具有**最大深度**的一片树叶  $v$ ,  $v$  有一个父亲  $u$ , 根据前述命题,  $T^*$  是满的二叉树, 所以  $v$  还有一个**兄弟**  $w$ .  
那么  $w$  是  $T^*$  的一片树叶。

证明: 用反证法, 如果  $w$  下面还有一片树叶  $w'$ , 那么  $\text{depth}(w') > \text{depth}(v)$ , 与  $v$  具有最大深度相矛盾。

## ■ 具体的算法步骤(Huffman算法):

To construct a prefix code for an alphabet  $S$ , with given frequencies:

If  $S$  has two letters then

Encode one letter using 0 and the other letter using 1

Else

Let  $y^*$  and  $z^*$  be the two lowest-frequency letters

Form a new alphabet  $S'$  by deleting  $y^*$  and  $z^*$  and

replacing them with a new letter  $\omega$  of frequency  $f_{y^*} + f_{z^*}$

Recursively construct a prefix code  $\gamma'$  for  $S'$ , with tree  $T'$

Define a prefix code for  $S$  as follows:

Start with  $T'$

Take the leaf labeled  $\omega$  and add two children below it  
labeled  $y^*$  and  $z^*$

Endif

- 定理: 对于给定字母表得到的Huffman码就是最优前缀码。

- 证明：归纳步骤中用反证法。假设贪心算法产生的树 $T$ 不是最优的，存在树 $Z$ ，使得 $ABL(Z) < ABL(T)$ ，根据前面的命题，存在这样一棵树 $Z$ ，其中 $y^*$ 与 $z^*$ （频率最低的两个字母）是兄弟。类似的，我们可以得到 $Z'$ ，满足 $ABL(Z') = ABL(Z) - f_w$ 。于是就得到 $ABL(Z') < ABL(T')$ ，与作为 $S'$ 前缀码的 $T'$ 最优性相矛盾。
- 实现与运行时间
- 一般的代价
  - 识别最低频率的字母在 $O(k)$ 时间内，
  - 迭代求和总的代价为 $O(k^2)$
- 采用优先队列(用堆实现)
  - 每次插入和最小元素的取出调整时间 $O(\log k)$ ，
  - 总运行时间 $O(k \log k)$

作业&答案：

## CH05：分治策略

分治算法思想：分成两个子问题，再递归解决子问题，时间复杂度由  $O(n^2)$  变成  $O(n \log n)$ 。应用：计数逆序（一分为二，重点是融合）、临近点对、整数乘法、矩阵乘法

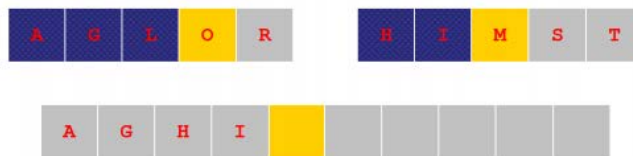
- 把一个规模为 $n$ 的问题分解成两个规模 $\frac{1}{2}n$ 的问题；
- 分别递归解决两个部分的问题；
- 在线性时间内把这两个部分问题的解合成一个完整的解。



## 1、归并排序（没重点说这个，去年考的这个）

### 归并排序

- 合并：把两个排好序的部分合成一个完整的排序
- 效率分析：采用一个临时数组；需要 $O(n)$ 次比较



- 对于归并排序算法，设 $T(n)$ 表示该算法在规模为 $n$ 的输入实例上最坏的运行（比较）时间。
- 开始假设 $n$ 是2的整数次幂
- 命题5.1 对某个常数 $c$ ,

$$T(n) \leq \begin{cases} 2T(n/2) + cn, & n > 2 \\ c, & n = 2 \end{cases}$$

- 或者

$$T(n) \leq \begin{cases} 0 & \text{if } n=1 \\ \underbrace{T(\lceil n/2 \rceil)}_{\text{solve left half}} + \underbrace{T(\lfloor n/2 \rfloor)}_{\text{solve right half}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

证明：

a)  $n$  是 2 的整数次幂：



- 命题：如果 $T(n)$ 满足如下关系，且 $n$ 是2的整数次幂，那么 $T(n) = n \log_2 n$ .

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

- 证明：对于 $n > 1$ ,可以得到

$$\begin{aligned} \frac{T(n)}{n} &= \frac{2T(n/2)}{n} + 1 \\ &= \frac{T(n/2)}{n/2} + 1 \\ &= \frac{T(n/4)}{n/4} + 1 + 1 \\ &\dots \\ &= \frac{T(n/n)}{n/n} + \underbrace{1 + \dots + 1}_{\log_2 n} \\ &= \log_2 n \end{aligned}$$

证明：用归纳法也可以：

$n=1$  命题显然成立；

假设 $T(n) = n \log_2 n$ 成立( $n = 2^k$ )，那么

$$\begin{aligned} T(2n) &= 2T(n) + 2n \\ &= 2n \log_2 n + 2n \\ &= 2n(\log_2(2n) - 1) + 2n \\ &= 2n \log_2(2n) \end{aligned}$$

所以命题对于 $n = 2^{k+1}$ 成立。

12

回到一般情形

命题：如果 $T(n)$ 满足如下关系，那么 $T(n) \leq n \lceil \lg n \rceil$ .

$$T(n) \leq \begin{cases} 0 & \text{if } n = 1 \\ \underbrace{T(\lceil n/2 \rceil)}_{\text{solve left half}} + \underbrace{T(\lfloor n/2 \rfloor)}_{\text{solve right half}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

## 递推关系

### ■ 采用部分替换的方法

在这种替换中，先猜想出解的形式，但是开始的时候常数项以及相关的参数不确定；在随后的推导中可以确定这些参数的值。有点类似于“待定系数法”。

## 其他递推关系

### ■ 命题5.3 对某个常数c,

$$T(n) \leq \begin{cases} qT(n/2) + cn, n > 2 \\ c, n = 2 \end{cases}$$

$q > 2, q = 1, q = 2$  递推关系的性质不同

$$\begin{aligned} T(n) &\leq \sum_{j=0}^{\log_2 n - 1} \left(\frac{q}{2}\right)^j cn = cn \sum_{j=0}^{\log_2 n - 1} \left(\frac{q}{2}\right)^j = cn \left(\frac{r^{\log_2 n} - 1}{r - 1}\right) \\ &\leq cn \left(\frac{r^{\log_2 n}}{r - 1}\right) = \frac{c}{r - 1} n \cdot r^{\log_2 n} = \frac{c}{r - 1} n \cdot n^{\log_2 r} = \frac{c}{r - 1} n^{\log_2 q} = O(n^{\log_2 q}) \end{aligned}$$

定理5.4 任何满足5.3并具有 $q > 2$ 的函数 $T(\cdot)$ 是  $O(n^{\log_2 q})$  有界的。

定理5.5 任何满足5.3式并具有 $q = 1$ 的函数 $T(\cdot)$  ( $T(n) \leq T(n/2) + O(n)$ ) 是  $O(n)$  有界的。

证明：

$$T(n) \leq \sum_{j=0}^{\log_2 n - 1} \frac{cn}{2^j} = cn \sum_{j=0}^{\log_2 n - 1} \frac{1}{2^j} \leq 2cn = O(n)$$

- 命题5.6 对某个常数c,

$$T(n) \leq \begin{cases} 2T(n/2) + cn^2, & n > 2 \\ c, & n = 2 \end{cases}$$

T(n)的复杂度应该是?

$$T(n) = O(n^2)$$

## 2、计数逆序



### ■ 分治策略

- 划分：把序列一分为二；
- 分别递归求出左右各自部分的逆序数；
- 组合：计数左右部份之间的逆序数, 返回总的逆序数。

1 5 4 8 10 2 6 9 12 11 3 7

Divide:  $O(1)$ .

1 5 4 8 10 2 6 9 12 11 3 7

Conquer:  $2T(n/2)$

5 blue-blue inversions

8 green-green inversions

9 blue-green inversions

5-3, 4-3, 8-6, 8-3, 8-7, 10-6, 10-9, 10-3, 10-7

Total =  $5 + 8 + 9 = 22$ .

Efficient Combine: ???



- 组合：计数蓝-绿逆序，**假设左右部分已经排序**，合并这两个部分为完整的排序。

3 7 10 14 18 19 2 11 16 17 23 25

6 3 2 2 0 0

13 blue-green inversions:  $6 + 3 + 2 + 2 + 0 + 0$

Count:  $O(n)$

2 3 7 10 11 14 16 17 18 19 23 25

Merge:  $O(n)$

$$T(n) \leq T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + O(n) \Rightarrow T(n) = O(n \log n)$$

## 算法

### Merge-and-Count(A,B)

维护一个Current指针指向每个表，初始化指向首元素

维护一个变量Count用于逆序的计数，初始为0

While两个表都不空

    令 $a_i$ 与 $b_j$ 是由Current指针指向的元素

    把这两个表中较小的元素加到输出表中

    If  $b_j$ 是较小的元素 then

        把Count加上在A中剩余的元素数

    Endif

    把较小元素选出的表中的Current指针前移

Endwhile

一旦一个表为空，把另一个表剩余的所有元素加到输出中

返回Count和合并后的表

## 算法

### Sort-and-Count(L)

If这个表中有一个元素, then

    没有逆序

Else

    把这个表划分成两半,

    A包含前 $\lceil n/2 \rceil$ 个元素

    B包含剩下的 $\lfloor n/2 \rfloor$ 个元素

$(rA,A)=\text{Sort-and-Count}(A)$

$(rB,B)=\text{Sort-and-Count}(B)$

$(r,L)=\text{Merge-and-Count}(A,B)$

Endif

返回 $r=rA+rB+r$ 以及排好序的表L

- 定理5.7 Sort-and-Count算法正确对输入表排序并且计数逆序个数；它对具有 $n$ 个元素的表运行在 $O(n \log n)$ 时间。

### 3、最近邻点对

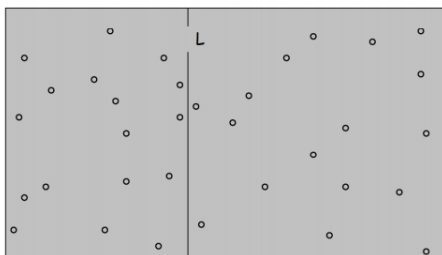
给定平面上的  $n$  个点，找最近邻的一对点。

|                                                                                                                                                           |               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| Closest-Pair( $p_1, \dots, p_n$ ) {                                                                                                                       |               |
| Compute separation line $L$ such that half the points are on one side and half on the other side.                                                         | $O(n \log n)$ |
| $\delta_1 = \text{Closest-Pair}(\text{left half})$                                                                                                        |               |
| $\delta_2 = \text{Closest-Pair}(\text{right half})$                                                                                                       | $2T(n/2)$     |
| $\delta = \min(\delta_1, \delta_2)$                                                                                                                       |               |
| Delete all points further than $\delta$ from separation line $L$                                                                                          | $O(n)$        |
| Sort remaining points by y-coordinate.                                                                                                                    | $O(n \log n)$ |
| Scan points in y-order and compare distance between each point and next 11 neighbors. If any of these distances is less than $\delta$ , update $\delta$ . | $O(n)$        |
| return $\delta$ .                                                                                                                                         |               |
| }                                                                                                                                                         |               |

44

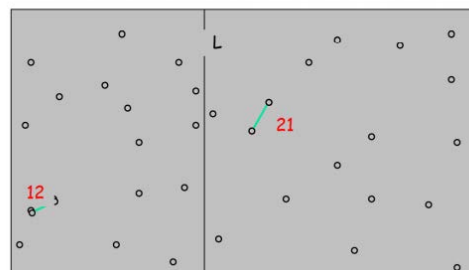
图示解释：

- 划分的时候：用一条垂直线把点集分成相等的两部分

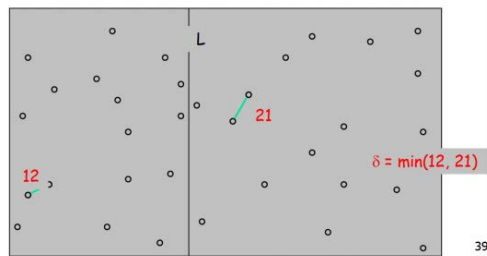
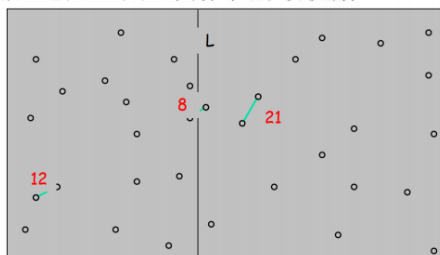


36

- 在左右半部分寻找各自最邻近的点对

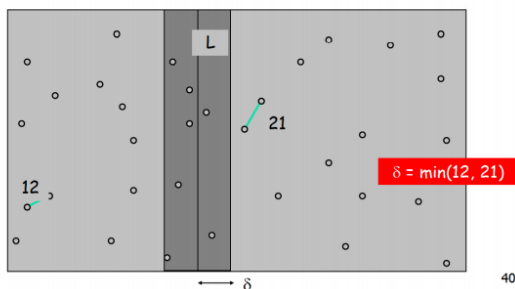


- 组合：在“交界”的边中寻找最近的点对
- 最后返回上面3个解中的最优解
- 问题的关键在于：寻找“交界”的边中比  $\delta = \min(\delta_L, \delta_R)$  还要近的点对。

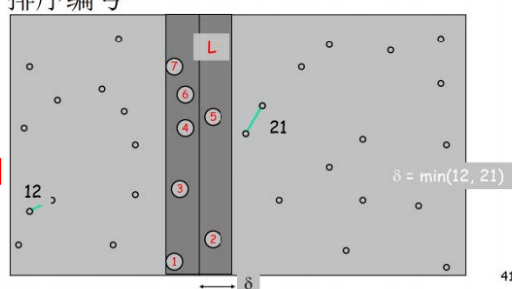


39

- 于是根据平面几何的约束性质，只需要考虑分界线附近的点集情况。
- 把“窄带”之中的点集按照y坐标的大小排序编号



40



41

定义 设 $s_i$  是 $2\delta$ 窄带中按照y坐标排序的第i个点。

命题. 如果  $|i - j| \geq 12$ , 那么 $s_i$ 与 $s_j$  之间的距离至少是 $\delta$ .

证明.

- 在 $\frac{1}{2}\delta \times \frac{1}{2}\delta$ 的小盒子中，至多只有一个点.
- $s_i$ 与 $s_j$  之间至少可以间隔两行，距离 $\geq 2(\frac{1}{2}\delta)$ .

运行时间：

$$T(n) \leq 2T(n/2) + O(n \log n)$$

复杂度？

$$T(n) = O(n \log^2 n)$$



## Closest-Pair-Rec (Px, Py)

If  $|P| \leq 3$  then

    进行简单穷举排序

End if

构造 $Q_x, Q_y, R_x, R_y$  ( $O(n)$ 时间)

$(q0^*, q1^*) = \text{Closest-Pair-Rec}(Q_x, Q_y)$

$(r0^*, r1^*) = \text{Closest-Pair-Rec}(R_x, R_y)$

$\Delta = \min(d(q0^*, q1^*), (r0^*, r1^*))$

$x^* =$ 集合 $Q$ 中的点的最大的 $x$ 坐标

$L = \{(x, y) : x = x^*\}$

$S = P$ 中与 $L$ 相距在 $\Delta$ 之内的点集

构造 $S_y$  ( $O(n)$ 时间)

For每个点 $S_y$ 中的点 $s$ , 计算从 $s$ 到跟着 $S_y$ 的15 (12) 个点中每个点的距离

令 $s, s'$  是达到其中最小距离的点 ( $O(n)$ 时间)

If  $d(s, s') < \Delta$  then

    Return  $(s, s')$

Else if  $d(q0^*, q1^*) < d(r0^*, r1^*)$  then

    Return  $(q0^*, q1^*)$

Else

    Return  $(r0^*, r1^*)$

End if

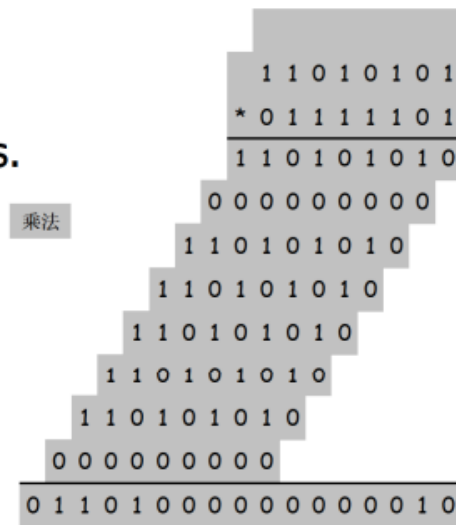
- 定理5.12 上面修改后的分治算法的运行时间是 $O(n \log n)$ .
- 证明:

$$T(n) \leq 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$$

### 3、整数乘法

## ■ 最直观的乘法

- $\Theta(n^2)$  bit operations.



52

## ■ Recursive-Multiply(x,y)

令  $x = x_1 2^{n/2} + x_0, y = y_1 2^{n/2} + y_0$

计算  $x_1 + x_0$  与  $y_1 + y_0$

$p = \text{Recursive-Multiply}(x_1 + x_0, y_1 + y_0)$

$x_1 y_1 = \text{Recursive-Multiply}(x_1, y_1)$

$x_0 y_0 = \text{Recursive-Multiply}(x_0, y_0)$

Return  $x_1 y_1 2^n + (p - x_1 y_1 - x_0 y_0) 2^{n/2} + x_0 y_0$

## ■ 定理5.13 [Karatsuba-Ofman, 1962]

Recursive-Multiply算法在两个n位因数上的运行时间是  $O(n^{\log_2 3}) = O(n^{1.59})$  .

## ■ 证明:

$$T(n) \leq \underbrace{T(\lfloor n/2 \rfloor) + T(\lfloor n/2 \rfloor) + T(1 + \lfloor n/2 \rfloor)}_{\text{recursive calls}} + \underbrace{\Theta(n)}_{\text{add, subtract, shift}}$$

$$\Rightarrow T(n) = O(n^{\log_2 3}) = O(n^{1.585})$$

## 4、 矩阵乘积

- 问题：给定两个 $n \times n$ 矩阵 $A, B$ , 求 $C=AB$ .

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj} \quad \begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}$$

- 如果采用直接计算，那么复杂度为 $\Theta(n^3)$  次算术运算(arithmetic operations).
- 存不存在更有效的办法？

### ■ 采用分治策略

- 先把 $A, B$ 分成 $\frac{1}{2}n \times \frac{1}{2}n$  的小块；
- 分别递归计算8块 $\frac{1}{2}n \times \frac{1}{2}n$ 矩阵乘积；
- 组合：把结果通过4次矩阵加法组合产生。

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \quad \begin{aligned} C_{11} &= (A_{11} \times B_{11}) + (A_{12} \times B_{21}) \\ C_{12} &= (A_{11} \times B_{12}) + (A_{12} \times B_{22}) \\ C_{21} &= (A_{21} \times B_{11}) + (A_{22} \times B_{21}) \\ C_{22} &= (A_{21} \times B_{12}) + (A_{22} \times B_{22}) \end{aligned}$$

$$T(n) = \underbrace{8T(n/2)}_{\text{recursive calls}} + \underbrace{\Theta(n^2)}_{\text{add, form submatrices}} \Rightarrow T(n) = \Theta(n^3)$$

### 矩阵快速乘积运算. (Strassen, 1969)

- 先把 $A, B$ 分成 $\frac{1}{2}n \times \frac{1}{2}n$  的小块；
- 计算：通过10次加减法运算，生成14 个 $\frac{1}{2}n \times \frac{1}{2}n$ 矩阵.
- 分治：递归生成7个 $\frac{1}{2}n \times \frac{1}{2}n$  的矩阵乘积.
- 组合：7个矩阵乘积通过8次矩阵的加减法生成需要的四个矩阵乘积

## ■ 分析

- 假设  $n$  是 2 的整数次幂.
- $T(n) = \# \text{ arithmetic operations.}$

$$T(n) = \underbrace{7T(n/2)}_{\text{recursive calls}} + \underbrace{\Theta(n^2)}_{\text{add, subtract}} \Rightarrow T(n) = \Theta(n^{\log_2 7}) = O(n^{2.81})$$

注意到这是一个理论上的界， $n$  比较大的时候才能体现出 Strassen 算法的优势；比如， $n \sim 2,500$ ，Strassen 算法在苹果机上的运行速度是常规算法的八十多倍

62

作业&答案：

## CH06：动态规划

递推公式、背包问题（有  $w$  限制）、分段的最小二乘（有枚举）、

RNA 的递推公式（序列比对不考）

对于带权区间调度问题，我们设计了一个多项式时间的算法：先设计一个（指数时间的）递归算法，然后通过备忘录将其转换成一个有效的递推算法，这个算法利用了全局数组  $M[]$  来记录递归函数值。

提出另外一种解决思路：在子问题上迭代，而不是递归的计算解

- 对于带权区间调度问题，我们可以不使用备忘录式的递归方法，而通过**迭代算法直接计算** $M[]$ 中的项。

```
Input:  $n, s_1, \dots, s_n, f_1, \dots, f_n, v_1, \dots, v_n$ 

Sort jobs by finish times so that  $f_1 \leq f_2 \leq \dots \leq f_n$ .

Compute  $p(1), p(2), \dots, p(n)$ 

Iterative-Compute-Opt
{
     $M[0] = 0$ 
    for  $j = 1$  to  $n$ 
         $M[j] = \max(v_j + M[p(j)], M[j-1])$ 
}
```

自底向上

28

## 动态规划类型问题:

1. 只存在多项式个子问题;
2. 容易从子问题的解计算初始问题的解;
3. 在子问题从“最小”到“最大”存在一种自然的顺序, 与一个容易计算的**递推式**相联系, 这个递推式允许从某些更小的子问题来确定一个子问题的解。

## 动态规划原理

### 动态规划类型问题:

1. 只存在多项式个子问题;
2. 容易从子问题的解计算初始问题的解;
3. 在子问题从“最小”到“最大”存在一种自然的顺序, 与一个容易计算的**递推式**相联系, 这个递推式允许从某些更小的子问题来确定一个子问题的解。

## 动态规划的过程

1. 刻画最优解的结构;
2. 递归定义最优解的值;
3. 按照自底向上的方式或自顶向下记忆化的方式计算最优解的值并记录下求解的途径;
4. 按照求解途径给出最优解的形成过程

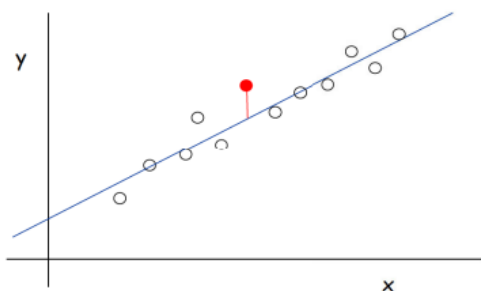
### 1、 分段最小二乘

## 分段的最小二乘

- 给定平面上的由 $n$ 个点组成的集合 $P: (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ .
- 假设 $x_1 < x_2 < \dots < x_n$
- 给定由方程 $y=ax+b$ 定义的直线 $L$ , 我们说 $L$ 相对于 $P$ 的误差是它对于 $P$ 中点的“距离”的平方和;
- 自然的目标是寻找具有最小误差的直线

### 误差

$$SSE = \sum_{i=1}^n (y_i - ax_i - b)^2$$





- 分段的最小二乘问题，问题形式化
- 给定一组点  $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ ，其中  $x_1 < x_2 < \dots < x_n$
- 寻找一组直线使得：在所有直线上的平方误差之和  $E$ ，直线的数目  $L$  之间找到一个好的平衡
  - 因此定义罚分度量函数： $E + cL$ ，其中  $c > 0$  是某一个常数。
- 采用动态规划的方法找到一个多项式时间的算法
- 定义.
  - $OPT(j)$  = 对于点  $p_1, p_{i+1}, \dots, p_j$  的最优解
  - $e(i, j)$  = 关系到  $p_i, p_{i+1}, \dots, p_j$  的任何直线的最小误差

递推式？

- 定理6.6 如果最优划分的最后一段是  $p_i, p_{i+1}, \dots, p_n$ ，那么最优解的值是  $OPT(n) = e(i, n) + c + OPT(i-1)$ .

为得到  $OPT(j)$ ，对于某一个  $i$ ，我们应该以最好的方式产生最后的一段  $p_i, p_{i+1}, \dots, p_j$ .

- 定理6.7 对于  $p_1, \dots, p_j$  的子问题，

$$OPT(j) = \begin{cases} 0 & \text{if } j = 0 \\ \min_{1 \leq i \leq j} \{ e(i, j) + c + OPT(i-1) \} & \text{otherwise} \end{cases}$$

现在设计这个算法中的困难部分已经克服：  
我们将给出一个自底向上的算法

```

INPUT:  $n, p_1, \dots, p_N, c$ 

Segmented-Least-Squares() {
     $M[0] = 0$ 
    for  $j = 1$  to  $n$ 
        for  $i = 1$  to  $j$ 
            compute the least square error  $e_{ij}$  for
                the segment  $p_i, \dots, p_j$ 

    for  $j = 1$  to  $n$ 
         $M[j] = \min_{1 \leq i \leq j} (e_{ij} + c + M[i-1])$ 

    return  $M[n]$ 
}

```

### 复杂度分析

- 算法的前半部分，计算 $O(n^2)$ 个 $e(i, j)$ ，根据前面给出的公式计算每一对 $e(i, j)$ 需要 $O(n)$ 的复杂度；  
后半部分，总的代价应该是 $O(n^2)$ ；
- 因此总的算法代价应该是 $O(n^3)$ ；如果预先计算了统计信息 $e(i, j)$ ，那么代价就是 $O(n^2)$ 。

## 算法

- 通过数组M向回寻找计算最优的划分

**Find-Segments(j)**

If  $j=0$  then

不用输出

Else

找一个使得 $e(i, j)+c+M[i-1]$ 最小的 $i$

输出这一段 $\{p_i, \dots, p_j\}$ 作为直线段，以及输出递归函数Find-Segments( $i-1$ )的结果

End if

### 2、子集和背包问题

给定 $n$ 个项 $\{1, \dots, n\}$ , 每个项有一个给定的非负的权 $w_i$ , 以及给定一个界 $W$ . 我们想选择项的一个子集 $S$ 使得 $\sum_{i \in S} w_i \leq W$ , 并且在这个前提下使得 $\sum_{i \in S} w_i$ 达到最大。这个问题称为子集和问题。

### Knapsack Problem

- 用物品 $\{1, \dots, n\}$ 的子集装入一个容量 $W$ 的背包使得它装的最满（或者装入的价值最大）。
- 每一个需求 $i$ 有一个值 $v_i$ 与一个权 $w_i$ , 在总权不超过 $W$ 的限制下，选择一个最大总值的子集。

例子：

问题：有一台可以处理作业的机器，我们有一组需求 $\{1,2,\dots,n\}$ .对于某个数 $W$ ,我们只能在时间0和时间 $W$ 之间的区间使用这个资源。每个需求对应于一个需要 $w_i$ 时间来处理的作业。我们的目标是处理这些作业，在“截至点” $W$ 之前能够使机器充分使用。

约束条件:  $\sum_{i \in S} w_i \leq W$

要求，最大化  $\sum_{i \in S} v_i$

例子:

$W = 11$

| Item | Value | Weight |
|------|-------|--------|
| 1    | 1     | 1      |
| 2    | 6     | 2      |
| 3    | 18    | 5      |
| 4    | 22    | 6      |
| 5    | 28    | 7      |

- 动态规划方法:
- 定义 $OPT(i, w)$  表示使用物品 $\{1,\dots,i\}$ 的子集且所允许的最大权是 $w$ 的最优解的值。
- ✓ 情形1  $OPT$  没有选择  $i$ ;  
 $OPT$  选择子集 $\{ 1, 2, \dots, i-1 \}$ , 采用 $w$  的最优解
- ✓ 情形2  $OPT$  选择  $i$ ;  
 余下部分 $OPT$  选择子集 $\{ 1, 2, \dots, i-1 \}$ , 采用 $w-w_i$  的最优解

- 据此马上我们就得到了递推式

$$OPT(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ OPT(i-1, w) & \text{if } w_i > w \\ \max \{ OPT(i-1, w), v_i + OPT(i-1, w-w_i) \} & \text{otherwise} \end{cases}$$

由此可见，增加一个变量重新进行动态规划，会变成更小的子问题，有助于设计动态规划算法。

- 自底向上的算法
- 填满一个  $n \times W$  的二维数组

```

Input: n, w1, ..., wN, v1, ..., vN

for w = 0 to W
    M[0, w] = 0

for i = 1 to n
    for w = 1 to W
        if (wi > w)
            M[i, w] = M[i-1, w]
        else
            M[i, w] = max { M[i-1, w], vi + M[i-1, w-wi] }

return M[n, W]

```

算法

|                 |   | 0 | 1 | 2 | 3 | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 |
|-----------------|---|---|---|---|---|----|----|----|----|----|----|----|----|
| φ               | 0 | 0 | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| {1}             | 0 | 1 | 1 | 1 | 1 | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  |
| {1, 2}          | 0 | 1 | 6 | 7 | 7 | 7  | 7  | 7  | 7  | 7  | 7  | 7  | 7  |
| {1, 2, 3}       | 0 | 1 | 6 | 7 | 7 | 18 | 19 | 24 | 25 | 25 | 25 | 25 | 25 |
| {1, 2, 3, 4}    | 0 | 1 | 6 | 7 | 7 | 18 | 22 | 24 | 28 | 29 | 29 | 40 | 40 |
| {1, 2, 3, 4, 5} | 0 | 1 | 6 | 7 | 7 | 18 | 22 | 28 | 29 | 34 | 34 | 40 | 40 |

OPT: { 4, 3 }  
value = 22 + 18 = 40

W = 11

| Item | Value | Weight |
|------|-------|--------|
| 1    | 1     | 1      |
| 2    | 6     | 2      |
| 3    | 18    | 5      |
| 4    | 22    | 6      |
| 5    | 28    | 7      |

56

## 时间复杂度?

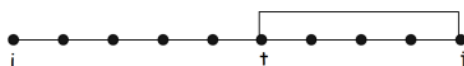
- 我们正在建立一个二位数组的表, 并且我们在 $O(1)$ 时间内计算每个值 $M[i,w]$ ; 于是得到
- 定理6.9 上面算法正确的计算这个问题的最优值, 并且运行在 $\Theta(n W)$ 时间。
- 定理6.10 给定子问题的最优值的表 $M$ , 可以在 $O(n)$ 时间内找到最优集合。
- 动态规划技术
  - 两种情况里面挑选最优解: 带权的区间调度.
  - 多种情况里面挑选最优解: 分段的最小二乘.
  - 增加一个变量: 背包问题.
  - 在区间上的动态规划: RNA 二级结构.
- 自顶向下(递归) vs 自底向上(循环): 不同的思路

### 3、 RNA 二级结构

对一个单螺旋RNA分子  $B = b_1b_2\dots b_n$ , 确定具有**最大碱基配对个数**的二级结构 $S$ .

#### 区间上的动态规划

- 定义 $OPT(i, j)$  表示在 $b_i b_{i+1} \dots b_j$ 上的二级结构中碱基对的最大数目。
  - 情形 1. 如果  $i \geq j - 4$ .
    - 根据没有尖的弯的规则,  $OPT(i, j) = 0$ .
  - 情形 2. 碱基 $b_j$  没有配对.
    - $OPT(i, j) = OPT(i, j-1)$
  - 情形 3. 碱基  $b_j$  与碱基  $b_t$  配对, 其中  $i \leq t < j - 4$ .
- 根据不交叉的限制就把问题分成两个子问题
  - $OPT(i, j) = 1 + \max_t \{ OPT(i, t-1) + OPT(t+1, j-1) \}$



69



- 定理 6.1.3

$OPT(i, j) = \max\{OPT(i, j-1), 1 + \max_t \{OPT(i, t-1) + OPT(t+1, j-1)\}\}$ ,

其中 $\max_t$ 遍取所有的 $t$ ,使得 $b_t$ 与 $b_j$ 是一对被允许的碱基.

- 确认我们建立子问题解的适当顺序
- 按照区间**长度增长**的顺序建立解

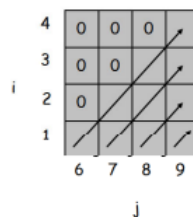
```

RNA( $b_1, \dots, b_n$ ) {
  for  $k = 5, 6, \dots, n-1$ 
    for  $i = 1, 2, \dots, n-k$ 
       $j = i + k$ 
      Compute  $M[i, j]$ 

  return  $M[1, n]$ 
}

```

\ using recurrence



- 算法时间复杂度
- 存在 $O(n^2)$ 个子问题;每次求值用 $O(n)$ 时间
- 总运行时间是 $O(n^3)$

作业&答案:

## CH07: 网络流

最大流算法（找剩余图，找一条增广路径，合并到原先图中，再找增广路径，直至剩余图  $s-t$  中不包括增广路径，找到最大流）。最大流=最小割，找最小割方法（找到最大流，从剩余图里找能到的所有点  $A$  剩下的是  $B$ ）。例子：二分匹配、不交路径。对于无向图相当

于每条边双向，也能找最大流

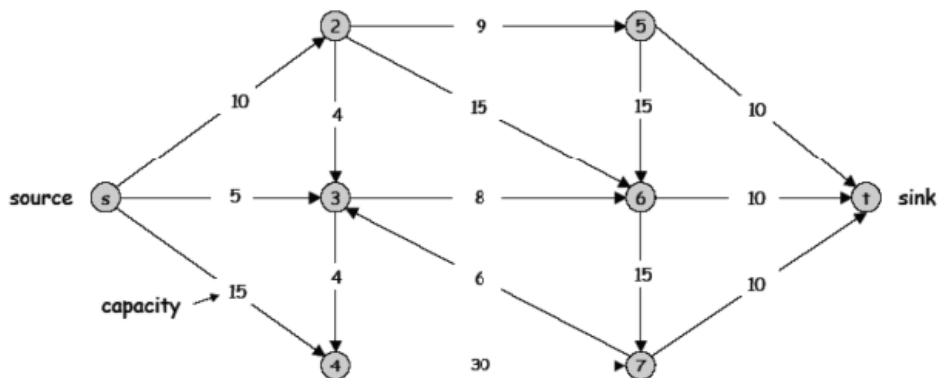
## 1、 最大流算法

### ■ 流网络

有向图 $G=(V,E)$

每条边关联一个容量，非负数 $C_e$ .

存在单一源点 $s$ , 以及单一汇点 $t$



### ■ 流的定义:

■  $s$ - $t$ 流是一个函数 $f$ ,它把每条边 $e$ 映射到一个非负实数 $f: E \rightarrow R^+$ ,值 $f(e)$ 表示由边 $e$ 携带的流量，一个流 $f$ 必须满足下面两个性质:

1. (容量条件) $0 \leq f(e) \leq C_e$
2. (守恒条件)除了 $s, t$ 外，对每个结点 $v$ ,满足
$$\sum_{e \in \text{in}_v} f(e) = \sum_{e \in \text{out}_v} f(e)$$

类似于一个从  $s$  到  $t$  的路径的权重和，在下面的应用中大概感觉结点间的流很

像是电流，能在节点进行汇总和重分配

■ **最大流问题**: 给定一个流网络，自然的目标就是安排交通使得有效容量尽可能得到有效使用：找出一个具有最大值的流

- 解决问题的思路

- 分解**

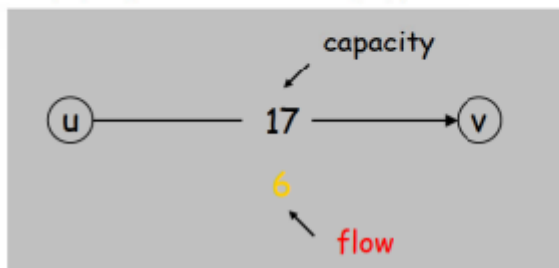
- 流的性质

- 我们可以在边上用剩余的容量**向前推**，并且我们可以在已经有流的边上**向后推**，使它转向一个不同的方向。

- 下面将引出**剩余图**的概念

- 原始边

$e = (u, v) \in E$ , 流  $f(e)$ , 容量  $c(e)$ .

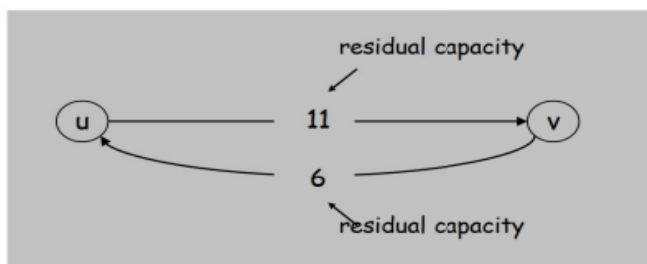


- **剩余边**

$e = (u, v)$  and  $e^R = (v, u)$ .

剩余容量:

$$c_f(e) = \begin{cases} c(e) - f(e) & \text{for } e \\ f(e) & \text{for } e^R \end{cases}$$



■ 剩余图:  $G_f = (V, E_f)$ .

- 具有正的剩余容量的剩余边.
- $E_f = \{e\} \cup \{e^R\}$ .

■ 对G的每条边 $e=(u,v)$ ,其中 $f(e) < c(e)$ ,那么存在 $c(e)-f(e)$ 的剩余的容量,我们还可以尝试在这个容量往前推,于是 $G_f$ 中包含这条边 $e$ ,容量为 $c(e)-f(e)$ ,称为前向边。

■ 对G的每条边 $e=(u,v)$ ,其中 $f(e) > 0$ ,我们可以通过向后推这个流来“撤销”它,于是 $G_f$ 中包含边 $e'=(v,u)$ ,容量是 $f(e)$ ,称为后向边。

■ 在剩余图中的增广路径

■ 令P是 $G_f$ 中一条简单的s-t路径。定义  $bottleneck(P, f)$  是P上任何边关于流f的 最小剩余容量。如下算法  $augment(f, P)$  在G中产生一个新的流f'.

```

Augment(f, P) {
  b ← bottleneck(P)
  foreach e ∈ P {
    if (e ∈ E) f(e) ← f(e) + b
    else      f(eR) ← f(e) - b
  }
  return f
}

```

前向边

后向边

- 通常把剩余图中的任何一条**s-t**路径认为是一条**增广路径**。
- 命题7.1  $f'$ 是**G**中的一个流。
- 证明：验证容量条件与守恒条件。  
 对于前向边： $0 \leq f(e) \leq f'(e) = f(e) + bottleneck(P, f) \leq C_e$   
 对于后向边： $c_e \geq f(e) \geq f'(e) = f(e) - bottleneck(P, f) \geq 0$

守恒条件：分情形讨论

- 增广操作保持了向前和向后流的守恒性
- 直觉上告诉我们，可以不断调整**G<sub>f</sub>**来获取更大的流量。

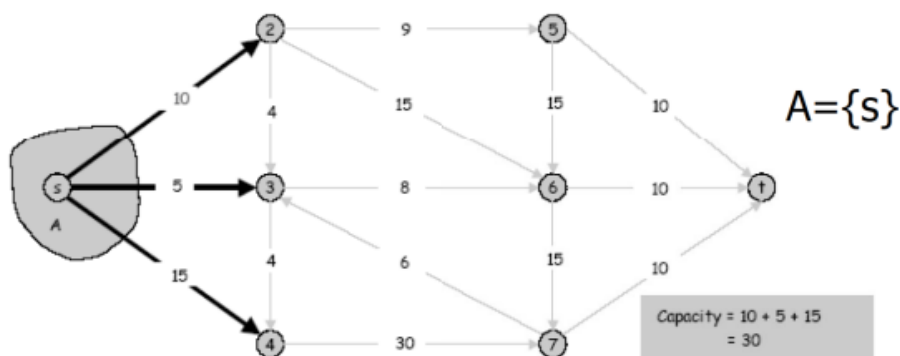
```
Ford-Fulkerson(G, s, t, c) {
    foreach e ∈ E  f(e) ← 0
    Gf ← residual graph

    while (there exists augmenting path P)
    {
        f ← Augment(f, c, P)
        update Gf
    }
    return f
}
```

22

## 2、 最大流和最小割

- 我们说一个**s-t割**是结点集合V的一个划分(A,B), 使得  $s \in A, t \in B$ . 一个割(A,B)的**容量**记为 $c(A,B)$ . 也就是从A出来的所有边的容量之和。



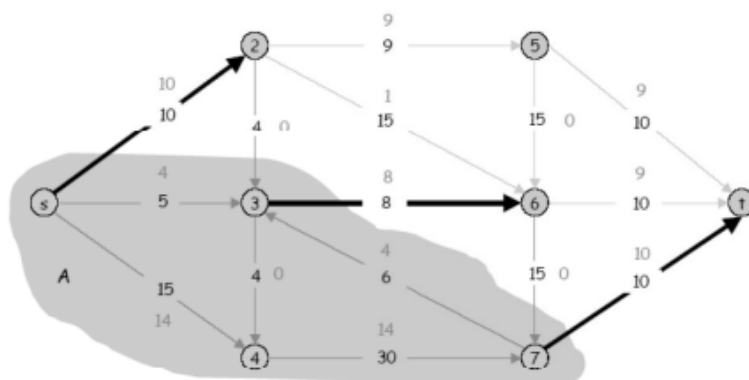
29

- 割原来提供了流值上的非常自然的上界
- 定理7.6 令 $f$ 是任何**s-t流**, 且(A,B)是任意**s-t割**, 那么  $v(f) = f^{out}(A) - f^{in}(A)$ .
- 命题7.7 令 $f$ 是任意**s-t流**, 且(A,B)是任意**s-t割**, 那么  $v(f) = f^{in}(B) - f^{out}(B)$



推论. 设 $f$ 是任意的流, 并设  $(A, B)$  是任意的割. 如果 $v(f) = \text{cap}(A, B)$ , 那么  $f$  是最大流, 并且 $(A, B)$  是最小割.

Value of flow = 28  
Cut capacity = 28  $\Rightarrow$  Flow value  $\leq$  28

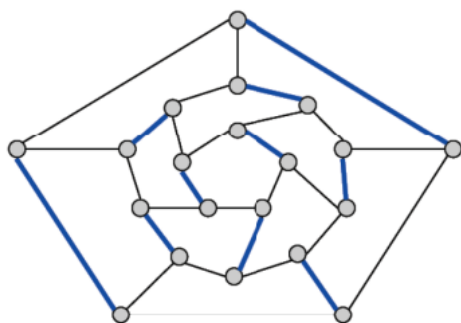


40

- 令  $\bar{f}$  表示由Ford-Fulkerson返回的流
- 下面我们将给出一个s-t割 $(A^*, B^*)$ 使得  $v(\bar{f}) = c(A^*, B^*)$   
这直接说明  $\bar{f}$  有任何流的最大值, 并且  $(A^*, B^*)$  有任何s-t割最小的容量
- 定理7.9 如果 $f$ 是使得剩余图 $G_f$ 中没有s-t路径的一个s-t流, 那么在 $G$ 中存在一个s-t割 $(A^*, B^*)$ 使得 $v(f) = c(A^*, B^*)$ . 因此,  $f$  有 $G$ 中任何流的最大值, 且 $(A^*, B^*)$ 有 $G$ 中任何s-t割的最小容量。
- 最大流最小割定理. [Ford-Fulkerson 1956] 最大流的值等于最小割

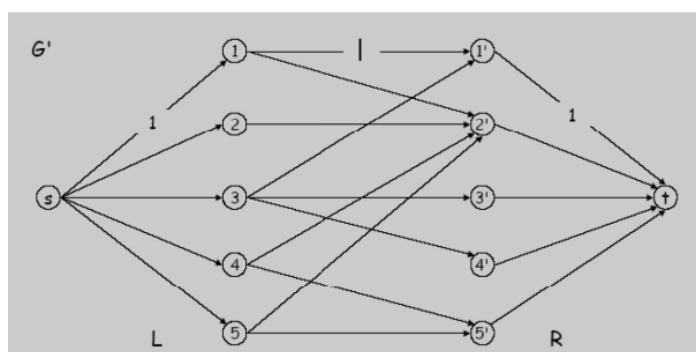
## 7.5 二分匹配问题

- 输入：无向图  $G = (V, E)$ .
- $M \subseteq E$  是一个匹配，如果每个结点至多出现在  $M$  中的一条边中。
- 最大匹配：寻找具有最大数目的匹配



65

- **二部图**  $G=(V,E)$  是一个无向图，它的结点集合可以被划分成  $V= L \cup R$ , 并具有下述性质：每条边  $e \in E$  有一个端点在  $L$  中，另一个端点在  $R$  中。
- 二分匹配。
  - 输入：无向，二部图,  $G = (L \cup R, E)$ .
  - $M \subseteq E$  是一个**匹配**，如果每个结点至多出现在  $M$  中的一条边中。
  - 最大匹配：寻找具有最大数目的匹配
- 最大流的构造。
  - 构造图  $G' = (L \cup R \cup \{s, t\}, E')$ .
  - 连接原图  $L$  到  $R$  的每条边, 每条边赋予单位容量.
  - 增加一个源点  $s$ , 从  $s$  到  $L$  中的每个结点连接一条边, 每条边赋予单位容量.
  - 增加一个终点  $t$ , 从  $R$  中的每个结点到  $t$  连接一条边, 每条边赋予单位容量.



- 现在计算这个网络 $G'$ 的最大 $s$ - $t$ 流，我们发现这个流的值等于 $G$ 中最大匹配的大小。

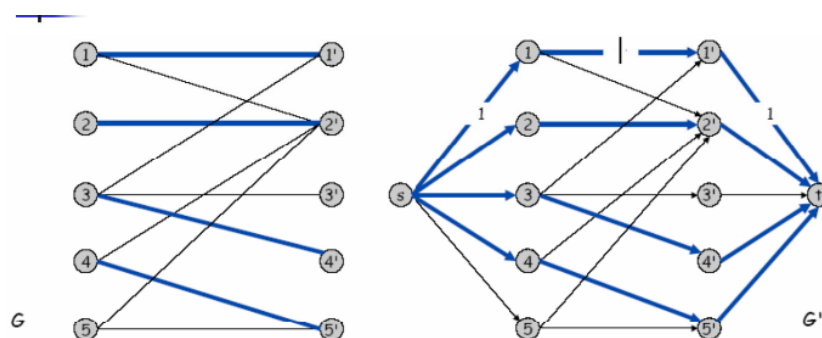
- 定理.  $G$ 中最大匹配的数目与所定义的 $G'$ 中最大流值相同.

- 证明:

设 $G$ 中最大匹配集合是 $M$ ，其数目是  $k$ .

- 于是可以构造一个流 $f$ ，每一条边从 $s$ 出发，携带一个单位的容量.
- $f$  是一个流，而且流值为 $k$ .
- 所以 $G'$ 中最大流值  $\geq$  最大匹配数目;

72



设  $f$  是  $G'$  最大流，其值为 $k$ .

- 考虑集合 $M$ : 从 $L$ 到 $R$ 权值为1的边的集合,  
i.e.,  $f(e) = 1$ .

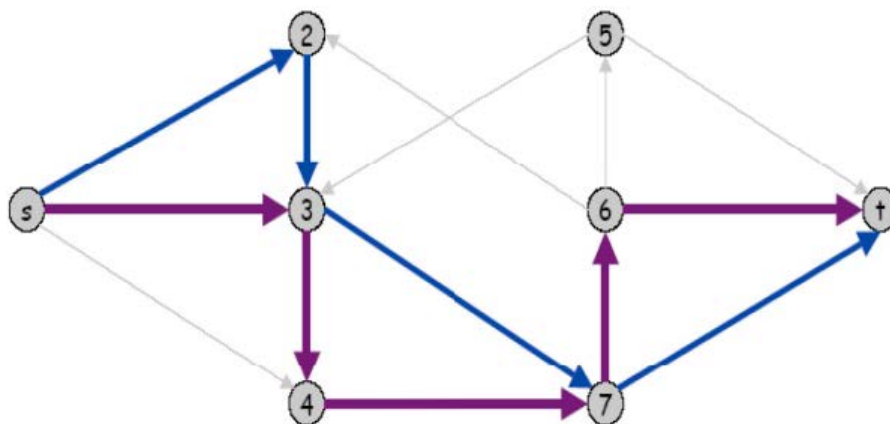
- 可以发现每个节点至多在 $M$ 的一条边中
- $|M| = k$ : 割  $(L \cup s, R \cup t)$  就是一个匹配

所以最大匹配的数目  $\geq$  最大流值

因此定理成立。

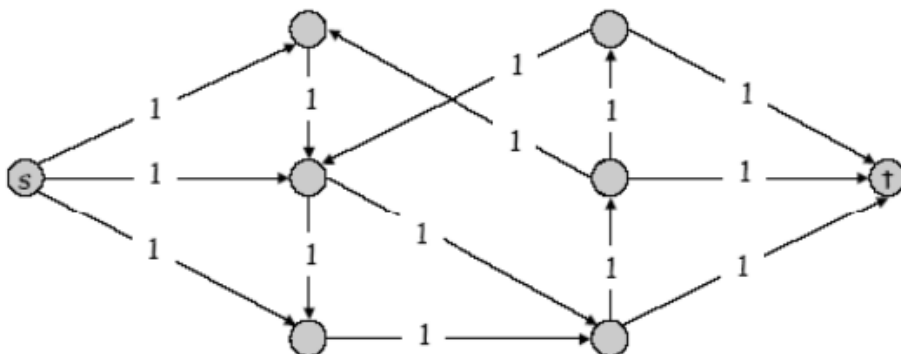
#### 4、 不交路径

- 定义 两条路径称为是**不交路径**，如果它们没有公共边。
- 不交路径问题. 给定图  $G = (V, E)$  以及两个结点  $s, t$ , 寻找最大的不相交  $s$ - $t$  路径数目.



不交路径的最大数目：2

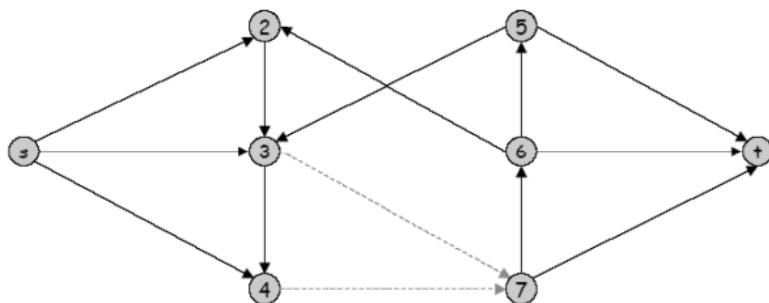
- 给定图  $G=(V,E)$  以及不同的结点  $s, t$ , 定义一个流网络：其中  $s, t$  分别是源点，汇点，且每条边容量为1.



- 假设存在 $k$ 条边不交的 $s$ - $t$ 路径，我们可以使每一条这样的路径携带一个单位的流；
- 对任何这种路径的边 $e$ 设定 $f(e)=1$ ,其他的边 $f(e')=0$ .
- 命题7.41 如果在有向图 $G$ 中从 $s$ 到 $t$ 存在 $k$ 条边不交的路径，那么 $G$ 中的最大 $s$ - $t$ 流至少是 $k$ .
- 最大流 $f$ 是整数值，而且所有的边容量以1为界，因此每条在 $f$ 下携带流的边上恰好有一个单位的流
- 命题7.42 如果 $f$ 是值为 $v$ 的0-1流，那么具有流值 $f(e)=1$ 的边的集合包含一组 $v$ 条(边不交)路径。
- 定理7.43 在有向图 $G$ 中从 $s$ 到 $t$ 存在 $k$ 条边不交路径当且仅当在 $G$ 中 $s$ - $t$ 流的最大值至少是 $k$ .
- 运行时间
- $C \leq |V| = n$ , 因此利用一般的Ford-Fulkerson算法可以在 $O(mn)$ 时间内得到一个整数最大流。
- 定理7.44 可以用Ford-Fulkerson算法在 $O(mn)$ 时间内在有向图 $G$ 中找到一组最大的边不交的 $s$ - $t$ 路径。

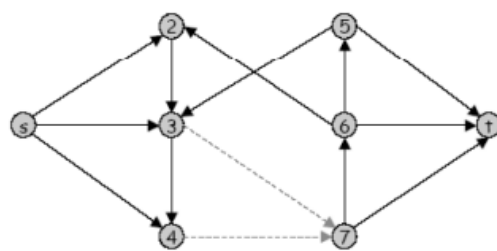
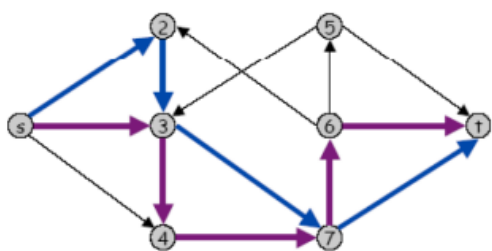


- 如果从图 $G$ 中拿走一组边 $F \subseteq E$ 以后，图中一条 $s$ - $t$ 路径也没剩下，我们就说 $F$ 分离 $s$ 与 $t$ 。(所有 $s$ - $t$ 路径至少使用 $F$ 中的一条边)



86

- 定理7.45 [Menger 1927] 在每个具有结点 $s$ 与 $t$ 的有向图中，边不交 $s$ - $t$ 路径的最大数目等于为分离 $s$ 与 $t$ 所需移走的最少边数。
- 证明：如果移走边集 $F \subseteq E$ 就分离了 $s$ 与 $t$ ，那么边不交的 $s$ - $t$ 路径数目至多是 $|F|$ ，于是边不交 $s$ - $t$ 路径的最大数目  $\leq$  分离 $s$ 与 $t$ 所需移走的最少边数；



87

## 5、 无向图的不交路径

实际上将无向图的边当作是双向的有向边。

## 无向图的不交路径

考虑无向图的情形

- 仍然可以借鉴有向图的想法
- 把 $G$ 中的每条无向边 $(u,v)$ 用两条有向边 $(u,v), (v,u)$ 代替
- 注意两条路径在有向图中可能是边不交的，但是无向图中却可能共享一条边



- 类似的我们可以采用有向图中的**Ford-Fulkerson**算法，以及路径分解过程来得到无向图中的边不交路径。
- **定理7.47** 在无向图 $G$ 中存在 $k$ 条从 $s$ 到 $t$ 的边不交的路径，当且仅当在 $G$ 对应的有向图 $G'$ 中 $s$ - $t$ 流的最大值至少是 $k$ . 此外，可以用**Ford-Fulkerson**算法在 $O(mn)$ 时间内找到无向图 $G$ 中的一组最大的边不交的 $s$ - $t$ 路径。
- **定理7.48** 在每个具有节点 $s$ 与 $t$ 的无向图中，边不交的 $s$ - $t$ 路径的最大数目等于为分离 $s$ 与 $t$ 所需要移走的最少边数。